Semantic Search

Multimedia Retrieval · Chapter 5

Multimedia Retrieval - 2026 - Uni Basel

Roger Weber

Contents

5 - Semantic Search **1**
The Evolution of Semantic Representations 1
Where Semantic Search Fits in the Pipeline 2
How This Chapter Is Organized 2
Latent Semantic Indexing 4
Singular Value Decomposition 4
Folding In New Documents and Queries 5
Worked Example 6
Limitations of LSI 8
Word Embeddings 10
From LSI to Embeddings 10
Word2Vec 10
GloVe and Subword Models 13
Emergent Properties and Limitations 15
From Words to Sentences: Why Pooling Is Not Enough 16
Transformer Embeddings 18
Three Challenges of Word Embeddings for Neural Models 18
One-Hot Vectors and Why They Fail at Scale 18
Sub-Word Tokenization 19
Learned Embeddings 21
From Sequential to Parallel Context 21
BERT: Bidirectional Context 25
Semantic Similarity with Transformers 27
Bi-Encoders: Sentence-BERT 27
Cross-Encoders and Reranking 28
Late Interaction: ColBERT 29
Building Semantic Search 32
Embedding Model Selection 32
Pipeline Architectures 33
Efficiency at Scale: Multi-Stage Retrieval (Pipeline E) 35
Practical Recommendations 36
Summary 37
Model Comparison 37
Key Takeaways 37
Key Formulas 38
Self-Check Questions 39
Further Reading 39

5 - Semantic Search

A user searching for "heart disease" expects to find documents about myocardial infarction, coronary artery conditions, and cardiology treatment, even when those documents share no keywords with the query. Classical retrieval models based on BM25 or the vector space model treat terms as independent symbols: they can match "heart" to "heart" but not to "cardiac". Stemming and lemmatization help at the surface level, and hand-curated ontologies can map synonyms, but these approaches are labor-intensive, language-specific, and break down quickly when facing the full diversity of natural language expression.

Semantic search solves this problem by representing text as dense vectors in a continuous space where geometric proximity encodes meaning. Two words that never co-occur in a query-document pair can still be recognized as related if their embeddings point in similar directions. This shift from discrete term matching to continuous meaning matching (Figure 5.1) is the central theme of this chapter.

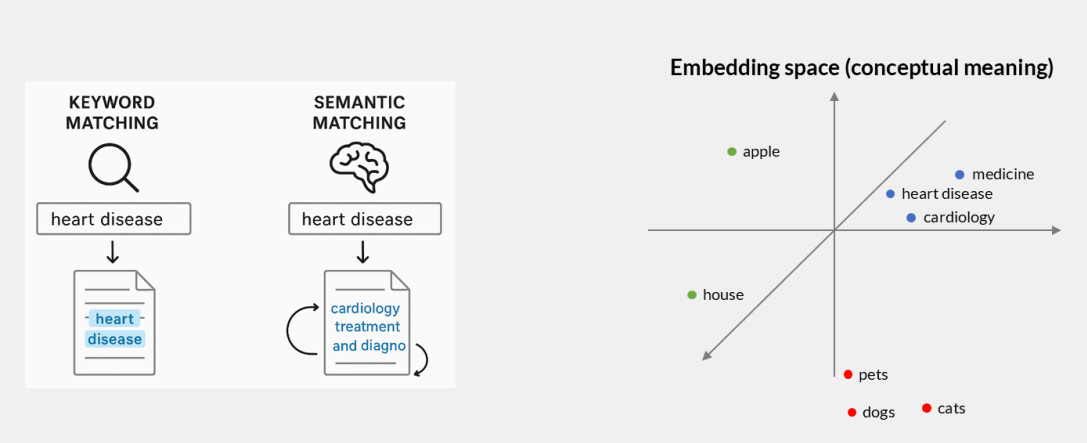


Figure 5.1: Keyword matching operates on exact surface forms (left). Semantic matching maps terms into a continuous embedding space where conceptually related terms cluster together (right), enabling retrieval even when query and document share no words.

The Evolution of Semantic Representations

The path from keyword retrieval to modern semantic search spans three decades and three paradigm shifts:

1. **Latent Semantic Indexing** (Deerwester et al., 1988) applied Singular Value Decomposition to the term-document matrix, discovering latent topics that link related terms. LSI proved that dimensionality reduction could capture semantic relationships, but its computational cost and inability to use inverted indices limited practical adoption.
2. **Static word embeddings** (Mikolov et al., 2013; Pennington et al., 2014) learned dense vector representations from local context windows. Word2Vec and GloVe showed that simple neural architectures trained on large corpora produce vectors where semantic similarity corresponds to geometric proximity. However, each word receives a single vector regardless of context: "bank" has the same representation whether it refers to a financial institution or a riverbank.
3. **Contextual embeddings and transformers** (Vaswani et al., 2017; Devlin et al., 2019; Reimers and Gurevych, 2019) introduced attention mechanisms that produce different representations for the same word depending on its surrounding context. Sentence-BERT restructured transformers into efficient bi-encoders suitable for large-scale retrieval. Subsequent work refined this paradigm: ColBERT (Khattab and Zaharia, 2020) introduced token-level late interaction for higher quality, and Matryoshka embeddings (Kusupati et al., 2022) enabled multi-resolution search. Production systems today typically combine sparse and dense retrieval in hybrid pipelines.

Where Semantic Search Fits in the Pipeline

Semantic search primarily improves **recall**: it expands the pool of candidate documents by finding matches that keyword methods miss. This comes at a cost. Dense embeddings lack efficient retrieval structures like inverted indices, so searching through millions of vectors requires approximate methods (covered in the next chapter). And because semantic retrieval casts a wider net, precision often drops; a reranker is typically needed to refine the ordering.

In practice, BM25 is not replaced by vector search. Instead, production systems combine both in hybrid pipelines. The blueprint is always the same two-component structure shown in Figure 5.2: a **retriever** that efficiently selects candidates (optimized for recall), followed by a **ranker** that reorders them (optimized for precision). What changes across pipelines is how each component is implemented.

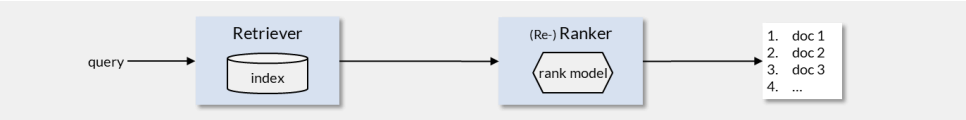


Figure 5.2: The retriever-ranker blueprint: a retriever selects candidate documents from the index (recall-focused), then a ranker applies deeper analysis to reorder them (precision-focused). Each pipeline below implements these two components differently.

Pipeline	Retriever	Ranker	Scale	Approach
A	BM25	BM25	Any	Fast keyword retrieval via inverted index. Misses semantic matches.
B	BM25	Cross-encoder	Small-medium	BM25 retrieves candidates, cross-encoder reranks top-k. Simple, effective.
C	Bi-encoder	Cross-encoder	Small	Vector search retrieves by semantic similarity, cross-encoder reranks. Good recall, misses exact matches.
D	BM25 + Bi-encoder	Cross-encoder	Medium-large	Both retrieve in parallel, results are fused, then cross-encoder reranks. Covers lexical and semantic matches.
E	Bi-encoder (compact)	Bi-encoder (full dims) + Cross-encoder	Very large	Cheap first stage (Matryoshka truncated dimensions or quantized embeddings), refined with full embeddings, then cross-encoder. Optimizes cost at billion-document scale.

How This Chapter Is Organized

This chapter covers the full progression from early matrix methods to production-ready semantic search systems:

- Latent Semantic Indexing introduces dimensionality reduction via SVD and shows how LSI discovers latent topics that connect related terms.

- Word Embeddings presents Word2Vec, GloVe, and subword models that learn distributional word representations from context.
- Transformer Embeddings explains sub-word tokenization, learned embeddings, the attention mechanism, and BERT's bidirectional contextual encoder. This section is optional background reading: it shows how contextual embeddings are produced, and a reader focused on retrieval can skim or skip it, since the next section opens with a short recap of the outcomes it relies on.
- Semantic Similarity with Transformers builds retrieval models on top of BERT: bi-encoders (SBERT), cross-encoders, and late interaction models (ColBERT).
- Building Semantic Search covers practical pipeline design: embedding model selection, document chunking, hybrid retrieval, and efficiency techniques.

The next chapter on Vector Search addresses the indexing problem: how to efficiently search through millions of embedding vectors using approximate nearest neighbor algorithms.

What you'll learn

- Explain how SVD-based dimensionality reduction captures latent semantic relationships between terms
- Compare static word embeddings (Word2Vec, GloVe) with contextual embeddings (BERT, SBERT) and articulate why context matters
- Analyze the tradeoffs between bi-encoders, cross-encoders, and late interaction models in terms of quality, latency, and scalability
- Design a multi-stage semantic search pipeline that combines sparse and dense retrieval with reranking
- Evaluate embedding models using standardized benchmarks (MTEB) and select appropriate models for a given use case

Prerequisites

- Classical Text Retrieval (Ch. 1): TF-IDF weighting, cosine similarity, the vector space model
- Advanced Text Processing (Ch. 3): tokenization, subword segmentation (BPE, WordPiece)
- Indexing for Text Retrieval (Ch. 4): inverted index, BM25 scoring
- Basic linear algebra: matrix multiplication, eigenvalues, orthogonality
- Basic neural networks: gradient descent, loss functions

Latent Semantic Indexing

In classical retrieval, documents are sparse high-dimensional vectors with TF-IDF weights, and similarity depends on shared terms. This term independence assumption limits recall: a document about "automobile maintenance" will not match a query for "car repair" unless we explicitly expand the query with synonyms. Stemming and lemmatization reduce some mismatches, but they are language-specific, require manual effort, and cannot capture domain-specific relationships. A search for "tokens" in a text processing context should also retrieve paragraphs discussing "terms" or "words", but no stemmer will make that connection.

Latent Semantic Indexing (LSI) addresses this by projecting the high-dimensional term-document space into a compact, lower-dimensional representation where semantically related terms are brought closer together. The key insight is that terms appearing in similar document contexts are likely related, even if they never co-occur in the same document. By reducing dimensionality, LSI forces the model to merge these co-occurrence patterns into latent topics.

LSI in historical context (optional reading)

LSI was developed at Bellcore (Bell Communications Research) in the late 1980s by a team that included Scott Deerwester and Susan Dumais. The first papers appeared in 1988, and a foundational patent was filed the same year and granted in 1989 (since expired). LSI demonstrated that unsupervised dimensionality reduction could improve retrieval quality, a foundational idea that influenced all subsequent embedding methods. Despite this conceptual contribution, LSI faced two practical barriers: the expensive SVD computation over large matrices, and the inability to use inverted indices for the resulting dense vectors. While computational costs have since become manageable, LSI has been superseded by neural embedding methods that capture finer-grained semantic associations and transfer across collections.

Singular Value Decomposition

Recall from the vector space model that we represent a collection as a term-document matrix $\boldsymbol{A}$ of size $m \times n$, where m is the vocabulary size and n the number of documents. Each entry a_{ij} holds the TF-IDF weight of term i in document j. Most entries are zero because any single document uses only a small fraction of the vocabulary, making $\boldsymbol{A}$ extremely sparse.

LSI builds on the Singular Value Decomposition (SVD), which factorizes this matrix into three components. Given $\boldsymbol{A}$ of rank r, the SVD produces:

$$\boldsymbol{A} = \boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^{\top} \tag{5.1}$$

where $\boldsymbol{U}$ is an $m \times r$ orthonormal matrix (each row is one term's representation in the r-dimensional latent space), $\boldsymbol{S}$ is an $r \times r$ diagonal matrix of singular values in decreasing order, and $\boldsymbol{V}$ is an $n \times r$ orthonormal matrix (each row is one document's representation in the latent space). Note that Figure 5.3 shows $\boldsymbol{V}^{\top}$ rather than $\boldsymbol{V}$, so the document representations appear as columns of $\boldsymbol{V}^{\top}$ (equivalently, rows of $\boldsymbol{V}$).

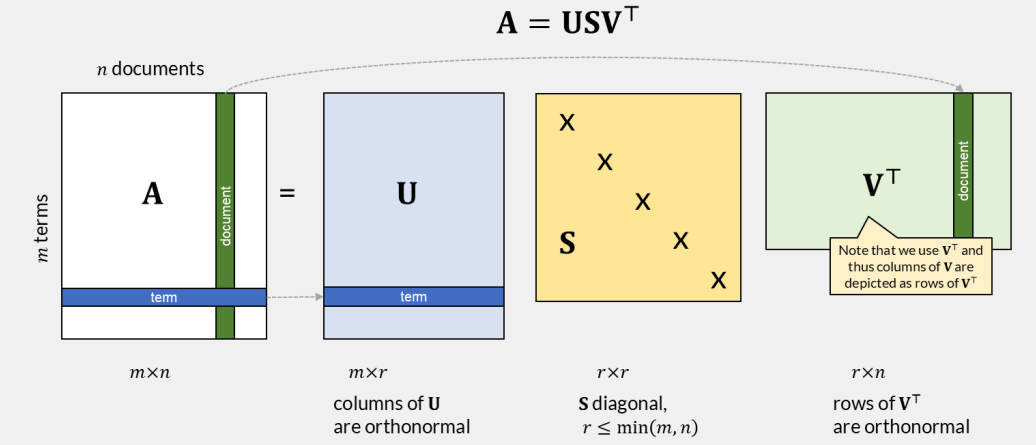


Figure 5.3: SVD decomposes the $m \times n$ term-document matrix $\boldsymbol{A}$ into three factors: $\boldsymbol{U}$ (term representations), $\boldsymbol{S}$ (singular values capturing topic importance), and $\boldsymbol{V}^\top$ (document representations).

The singular values on the diagonal of $\boldsymbol{S}$ decrease rapidly in magnitude. By retaining only the k largest singular values and their corresponding columns in $\boldsymbol{U}$ and $\boldsymbol{V}$, we obtain the best rank-k approximation of $\boldsymbol{A}$ under the Frobenius norm (Eigenvectors, SVD, and PCA shows why):

$$\boldsymbol{A}_k = \boldsymbol{U}_k \boldsymbol{S}_k \boldsymbol{V}_k^\top \tag{5.2}$$

The parameter k is a hyperparameter that controls a quality-storage tradeoff. The original matrix $\boldsymbol{A}$ has $m \times n$ entries. The truncated representation stores $\boldsymbol{U}_k$ ($m \times k$) and $\boldsymbol{V}_k$ ($n \times k$), plus the k singular values, totaling $(m + n + 1) \times k$ values. For typical collections where $k \ll \min(m, n)$, this is a large reduction. Choosing k too small loses semantic detail; choosing k too large retains noise and increases storage demand, and in our application, retrieval cost. In practice, values between 100 and 300 work well for text collections.

Figure 5.4 illustrates the truncated decomposition. Compared to Figure 5.3, the matrices are now much smaller: $\boldsymbol{U}_k$ has only k columns (one per latent topic) and $\boldsymbol{V}_k^\top$ has only k rows.

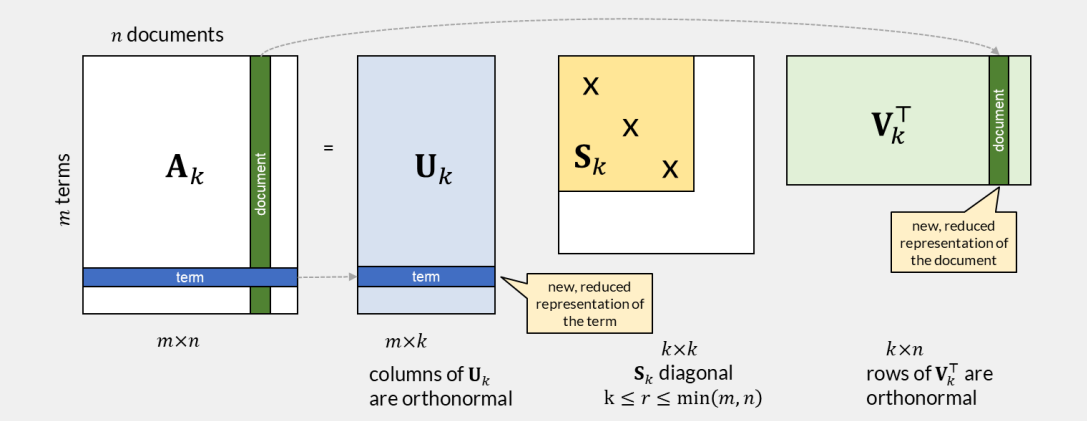


Figure 5.4: Truncated SVD retains only k dimensions. Rows of $\boldsymbol{U}_k$ are k-dimensional term vectors; columns of $\boldsymbol{V}_k^\top$ (rows of $\boldsymbol{V}_k$) are k-dimensional document vectors.

The k dimensions no longer correspond to individual terms. Each dimension represents a latent topic, a pattern of term co-occurrence across documents. We can interpret a topic by examining the terms with the largest weights in the corresponding column of $\boldsymbol{U}_k$.

Folding In New Documents and Queries

When a new document $\boldsymbol{d}$ arrives (represented as a term vector in the original m-dimensional space), we project it into the k-dimensional latent space without recomputing the full SVD (see Figure 5.5):

Key Formula: LSI Projection

$$\bar{\boldsymbol{d}}^\top = \boldsymbol{d}^\top \boldsymbol{U}_k \boldsymbol{S}_k^{-1} \tag{5.3}$$

A new document (or query) vector $\boldsymbol{d}$ is mapped to the latent topic space by multiplying with $\boldsymbol{U}_k \boldsymbol{S}_k^{-1}$. This is valid as long as the new document does not significantly alter the collection's topic structure.

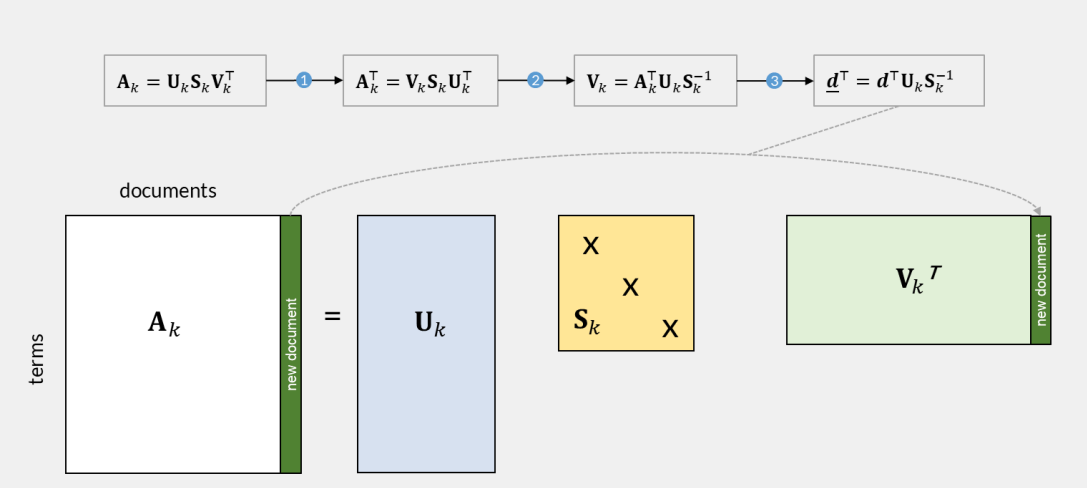


Figure 5.5: Derivation of the folding-in formula in three steps: transpose the truncated SVD equation, multiply by the pseudo-inverse, and isolate a single document vector.

Queries are treated as short documents. The query vector $\boldsymbol{q}$ (with non-zero entries for query terms) is projected identically:

$$\bar{\boldsymbol{q}}^\top = \boldsymbol{q}^\top \boldsymbol{U}_k \boldsymbol{S}_k^{-1} \tag{5.4}$$

Retrieval then computes cosine similarity between the projected query and all projected documents:

$$\text{sim}_{\cos}(Q, D_i) = \frac{\bar{\boldsymbol{q}} \cdot \bar{\boldsymbol{d}}_i}{|\,\bar{\boldsymbol{q}}\,| \cdot |\,\bar{\boldsymbol{d}}_i\,|} \tag{5.5}$$

Worked Example

Consider a collection of 9 documents split into two groups: five about human-computer interfaces (c1-c5) and four about graph algorithms (m1-m4):

ID	Title
c1	**Human** machine interface for Lab ABC **computer** applications
c2	A survey of user opinion of **computer** system response time
c3	The EPS user interface management system
c4	System and **human** system engineering testing of EPS
c5	Relation of user-perceived response time to error measurement
m1	The generation of random, binary, unordered trees
m2	The intersection graph of paths in trees
m3	Graph minors IV: Widths of trees and well-quasi-ordering
m4	Graph minors: A survey

A search for "**human computer interaction**" should retrieve c1-c5, but not every document contains those exact terms. Traditional retrieval requires explicit query expansion.

After constructing the term-document matrix (using term frequencies, excluding stop words and terms appearing only once), we compute the SVD and truncate to $k = 2$ topics (Figure 5.6).

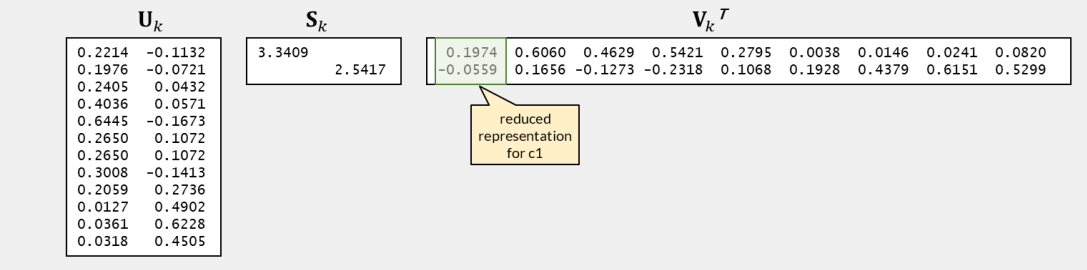


Figure 5.6: Truncated SVD with $k = 2$: the matrices $\boldsymbol{U}_k$ (12 terms × 2 topics), $\boldsymbol{S}_k$ (2 × 2), and $\boldsymbol{V}_k^\top$ (2 × 9 documents). Each document is now a 2D vector.

Only "human" and "computer" appear in the vocabulary, so the query is the sparse 12-dimensional vector with two active entries. Folding it into the 2D topic space with the projection formula yields $\bar{\boldsymbol{q}} = (0.138, \; -0.028)$, as shown in Figure 5.7.

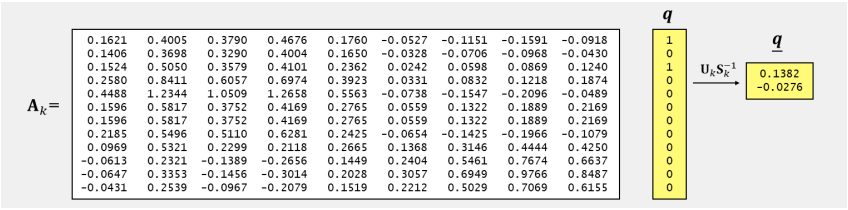


Figure 5.7: Folding-in of the query "human computer interaction": the sparse 12-dimensional term vector $\boldsymbol{q}$ is projected to a 2-dimensional latent representation via $\boldsymbol{U}_k\boldsymbol{S}_k^{-1}$.

Placing this query alongside the documents in the latent plane (see Figure 5.8) shows it pointing toward the c-documents:

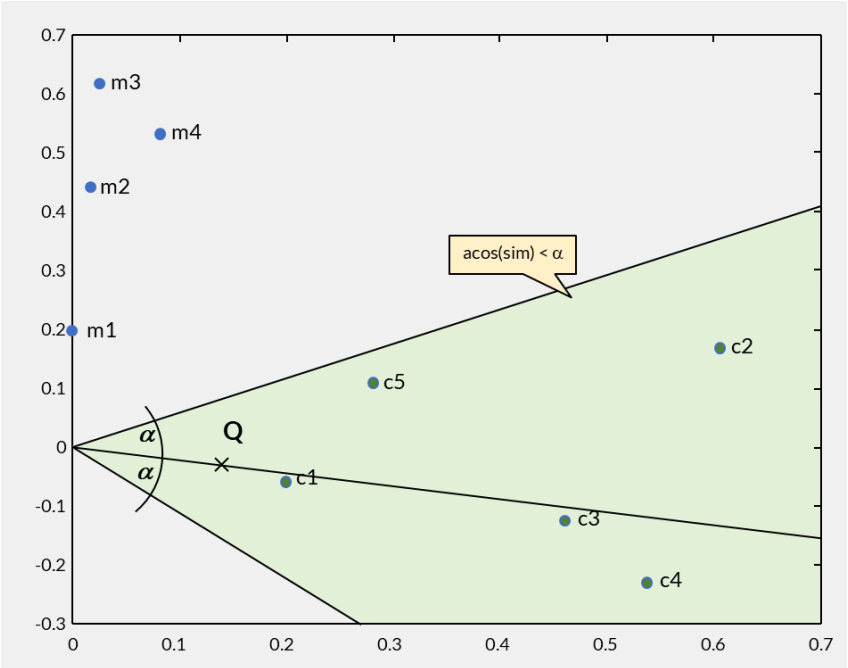


Figure 5.8: Documents and query in the 2D latent space. Topic 1 (horizontal) aligns with the interface documents c1-c5, while topic 2 (vertical) aligns with the graph documents m1-m4. The shaded angular cone shows all documents within angle α of the query vector.

Using cosine similarity, documents rank as: c3 (0.997) > c1 (0.997) > c4 (0.979) > c2 (0.895) > c5 (0.846), with all c-documents retrieved before any m-document (every m-document scores below zero). Notably, c3 ("The EPS user interface management system") ranks highest despite containing neither "human" nor "computer". The SVD has recognized that its terms ("EPS", "user", "interface", "system") co-occur with query-related terms in other documents, connecting them through the latent topic structure.

The two topics can be interpreted from the largest-weight terms in each column of $\boldsymbol{U}_k$:

- **Topic 1**: 0.64·system + 0.40·user + 0.30·EPS + 0.27·time + 0.27·response + 0.24·computer

- **Topic 2**: 0.62·graph + 0.49·trees + 0.45·minors + 0.27·survey

Sign ambiguity in SVD

The signs of the singular vectors are not unique. Flipping the sign of a column of $\boldsymbol{U}_k$ together with the matching row of $\boldsymbol{V}_k^\top$ gives an equally valid decomposition. Only the relative orientation of the latent coordinates carries meaning, so the absolute sign of a topic axis, and of individual coordinates such as the −0.028 above, depends on the SVD implementation. Cosine similarities are unaffected because query and documents are projected with the same $\boldsymbol{U}_k$.

Full numeric SVD matrices (optional reading)

Figure 5.9 lists the complete $\boldsymbol{U}$, $\boldsymbol{S}$, and $\boldsymbol{V}^\top$ matrices for this example, before truncation to $k = 2$. The nine singular values on the diagonal of $\boldsymbol{S}$ decrease from 3.34 to 0.36; keeping only the first two produces the rank-2 approximation used above. Rows 6 and 7 of $\boldsymbol{U}$ are identical because the terms "response" and "time" occur in exactly the same documents (c2 and c5), so the decomposition cannot separate them.

$$\mathbf{U} = \begin{bmatrix}
0.2214 & -0.1132 & 0.2890 & -0.4148 & -0.1063 & -0.3410 & 0.5227 & -0.0605 & -0.4067 \\
0.1976 & -0.0721 & 0.1350 & -0.5522 & 0.2818 & 0.4959 & -0.0704 & -0.0099 & -0.1089 \\
0.2405 & 0.0432 & -0.1644 & -0.5950 & -0.1068 & -0.2550 & -0.3022 & 0.0623 & 0.4924 \\
0.4036 & 0.0571 & -0.3378 & 0.0991 & 0.3317 & 0.3848 & 0.0029 & -0.0004 & 0.0123 \\
0.6445 & -0.1673 & 0.3611 & 0.3335 & -0.1590 & -0.2065 & -0.1658 & 0.0343 & 0.2707 \\
0.2650 & 0.1072 & -0.4260 & 0.0738 & 0.0803 & -0.1697 & 0.2829 & -0.0161 & -0.0539 \\
0.2650 & 0.1072 & -0.4260 & 0.0738 & 0.0803 & -0.1697 & 0.2829 & -0.0161 & -0.0539 \\
0.3008 & -0.1413 & 0.3303 & 0.1881 & 0.1148 & 0.2722 & 0.0330 & -0.0190 & -0.1653 \\
0.2059 & 0.2736 & -0.1776 & -0.0324 & -0.5372 & 0.0809 & -0.4669 & -0.0363 & -0.5794 \\
0.0127 & 0.4902 & 0.2311 & 0.0248 & 0.5942 & -0.3921 & -0.2883 & 0.2546 & -0.2254 \\
0.0361 & 0.6228 & 0.2231 & 0.0007 & -0.0683 & 0.1149 & 0.1596 & -0.6811 & 0.2320 \\
0.0318 & 0.4505 & 0.1411 & -0.0087 & -0.3005 & 0.2773 & 0.3395 & 0.6784 & 0.1825
\end{bmatrix}$$

$$\mathbf{S} = \begin{bmatrix}
3.3409 & & & & & & & & \\
& 2.5417 & & & & & & & \\
& & 2.3539 & & & & & & \\
& & & 1.6445 & & & & & \\
& & & & 1.5048 & & & & \\
& & & & & 1.3064 & & & \\
& & & & & & 0.8459 & & \\
& & & & & & & 0.5601 & \\
& & & & & & & & 0.3637
\end{bmatrix}$$

$$\mathbf{V}^\top = \begin{bmatrix}
0.1974 & 0.6060 & 0.4629 & 0.5421 & 0.2795 & 0.0038 & 0.0146 & 0.0241 & 0.0820 \\
-0.0559 & 0.1656 & -0.1273 & -0.2318 & 0.1068 & 0.1928 & 0.4379 & 0.6151 & 0.5299 \\
0.1103 & -0.4973 & 0.2076 & 0.5699 & -0.5054 & 0.0982 & 0.1930 & 0.2529 & 0.0793 \\
-0.9498 & -0.0286 & 0.0416 & 0.2677 & 0.1500 & 0.0151 & 0.0155 & 0.0102 & -0.0246 \\
0.0457 & -0.2063 & 0.3783 & -0.2056 & 0.3272 & 0.3948 & 0.3495 & 0.1498 & -0.6020 \\
-0.0766 & -0.2565 & 0.7244 & -0.3689 & 0.0348 & -0.3002 & -0.2122 & 0.0001 & 0.3622 \\
0.1773 & -0.4330 & -0.2369 & 0.2648 & 0.6723 & -0.3408 & -0.1522 & 0.2491 & 0.0380 \\
-0.0144 & 0.0493 & 0.0088 & -0.0195 & -0.0583 & 0.4545 & -0.7615 & 0.4496 & -0.0696 \\
-0.0637 & 0.2428 & 0.0241 & -0.0842 & -0.2624 & -0.6198 & 0.0180 & 0.5199 & -0.4535
\end{bmatrix}$$

Figure 5.9: The full (untruncated) SVD of the 12 × 9 term-document matrix: $\boldsymbol{U}$ (12 × 9), the diagonal singular values $\boldsymbol{S}$ (9 × 9) in decreasing order, and $\boldsymbol{V}^\top$ (9 × 9).

The accompanying demo notebook reproduces these matrices from the raw collection, visualizes the 2D document space interactively, and lets you experiment with different values of k.

Limitations of LSI

LSI demonstrated that latent semantic structure improves retrieval, but several limitations prevented widespread adoption:

1. **No inverted index acceleration**: projected document vectors are dense, so every query requires comparison against all documents. The pruning strategies that make inverted indices fast (early termination, skip pointers) do not apply.
2. **Corpus-specific topics**: the learned mapping depends on both terms and documents in the collection. An LSI model trained on IT articles performs poorly on biomedical text. Unlike modern embeddings, LSI representations do not transfer across domains.
3. **Static vocabulary**: new terms can only be approximated through the existing topic structure. Two new terms in the same document receive identical representations, since both depend solely on that document's projection into $\boldsymbol{V}_k$.

4. **No sub-word awareness**: LSI operates at the word level and cannot generalize to morphological variants or misspellings not already in the vocabulary.
5. **Linear assumption**: SVD captures linear relationships between terms and documents. Non-linear semantic patterns (metaphor, irony, context-dependent meaning) are beyond its reach.

Despite these limitations, LSI established the foundational principle that underpins all modern semantic search: by reducing dimensionality, we can discover latent structure that connects terms sharing similar usage patterns, even when they never co-occur directly. The methods that follow in this chapter, from Word2Vec to transformers, all build on this insight while addressing LSI's specific shortcomings.

Hands-on: Latent Semantic Indexing

Build the term-document matrix for the nine-document Deerwester collection, compute its SVD, truncate to two topics, and project the query "human computer interaction" to rank documents by cosine similarity in the latent space. Open notebook ->

Includes pre-run results. You can read through or download and experiment.

Word Embeddings

Word embeddings map each word in a vocabulary to a dense vector of d dimensions (typically 100-300), where words appearing in similar contexts receive similar vectors. Unlike LSI, which derives representations from a global term-document matrix, word embeddings learn from local context windows and scale efficiently to billions of training tokens. This section covers the foundational models: Word2Vec, GloVe, and fastText.

From LSI to Embeddings

While both LSI and word embeddings produce dense vector representations, they differ in fundamental ways:

Aspect	LSI	Word Embeddings
Unit of representation	Documents (and terms as by-product)	Words (documents via aggregation)
Context signal	Global co-occurrence across entire documents	Local context windows (typically 5-10 words)
Training	SVD on term-document matrix	Neural network with gradient descent
Vocabulary	Fixed; retraining needed for new terms	Sub-word variants possible (fastText)
Transferability	Corpus-specific; poor cross-domain	Language-level; transfers across collections
Scalability	SVD requires the full $m \times n$ matrix in memory; cubic in the smaller dimension	Streams through the corpus one window at a time; linear in corpus size

The key conceptual shift is the **distributional hypothesis**: a word's meaning is defined by the company it keeps (Firth, 1957). Word2Vec operationalizes this by training a neural network to predict either context from a center word (Skip-Gram) or a center word from its context (CBOW).

Word2Vec

Word2Vec, introduced by Mikolov et al. in 2013, maps words to a d-dimensional vector space using self-supervised learning on raw text. No labeled data is required. It has two architectures:

Skip-Gram

The Skip-Gram model operates within a context window of size $2m + 1$ around a center word. It learns to predict which words typically appear nearby a given center word. Consider the phrase "the dog chases a cat" with center word "chases" and $m = 2$ (a context window of 5 words in total). Skip-Gram estimates the probability that the center word has exactly these $2m = 4$ words in its context as a product of independent predictions, one factor for each surrounding word (see Figure 5.10):

$$P(\text{the, dog, a, cat} \mid \text{chases}) = P(\text{the} \mid \text{chases}) \cdot P(\text{dog} \mid \text{chases}) \cdot P(\text{a} \mid \text{chases}) \cdot P(\text{cat} \mid \text{chases}) \quad (5.6)$$

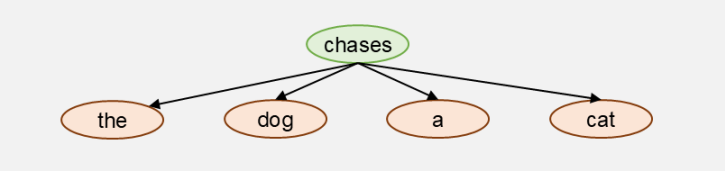


Figure 5.10: Skip-Gram context for "the dog chases a cat": the center word "chases" independently predicts each surrounding word within the window.

To compute these probabilities, Skip-Gram represents each word w_i with two d-dimensional vectors: $\boldsymbol{v}_i$ when it serves as a center word, and $\boldsymbol{u}_i$ when it serves as a context word. The semantic relationship between two words is captured by their inner product $\boldsymbol{u}_s^\top \boldsymbol{v}_c$: a large value means w_s is likely to appear near w_c, a small value means it is unlikely. To turn these raw scores into a probability distribution $P(w_s \mid w_c)$ over all vocabulary words $w_s \in \mathbb{T}$, we apply softmax:

$$P(w_s \mid w_c) = \frac{\exp(\boldsymbol{u}_s^\top \boldsymbol{v}_c)}{\sum_{i \in \mathbb{T}} \exp(\boldsymbol{u}_i^\top \boldsymbol{v}_c)} \tag{5.7}$$

To obtain the vectors $\boldsymbol{v}_i$ and $\boldsymbol{u}_i$, we maximize the product of $P(w_s \mid w_c)$ over all observed center-context pairs in the corpus. Taking the logarithm turns this product into a sum, giving us the equivalent objective of minimizing the negative log-likelihood over all windows:

Key Formula: Skip-Gram Loss

$$\mathcal{L} = -\sum_{i=m+1}^{n-m} \sum_{j=i-m,\, j \neq i}^{i+m} \log P(w_j \mid w_i) \tag{5.8}$$

The loss sums the negative log-probability of each context word given its center word, over all windows in the corpus. Minimizing this loss via gradient descent adjusts both $\boldsymbol{v}_i$ and $\boldsymbol{u}_i$ until the vectors predict observed co-occurrences well.

This is self-supervised learning: we extract training data directly from a large text corpus without manual labels. The corpus is first split into sentences, then each sentence is split into a sequence of terms, and overlapping context windows are generated strictly within sentence boundaries. This produces billions of training pairs from web-scale corpora. Preprocessing matters: punctuation, numbers, case normalization, and stop word handling all affect which pairs are generated and thus which relationships the model learns.

A consequence of the sentence-boundary constraint is that Word2Vec cannot connect concepts across sentences. Pronouns like “he”, “she”, or “it” that refer to entities introduced in a previous sentence receive vectors shaped only by their local window, not by their referent. This cross-sentence blindness is another limitation that contextual models later address.

We cannot solve this optimization in closed form. Instead, we use gradient descent on a neural network with a single hidden layer, as shown in Figure 5.11. The center word enters as a one-hot vector, the first weight matrix contains the center word vectors $\boldsymbol{v}_i$, and the second weight matrix contains the context word vectors $\boldsymbol{u}_i$. A softmax output layer produces the probability distribution over the vocabulary, compared against the actual context word via cross-entropy loss.

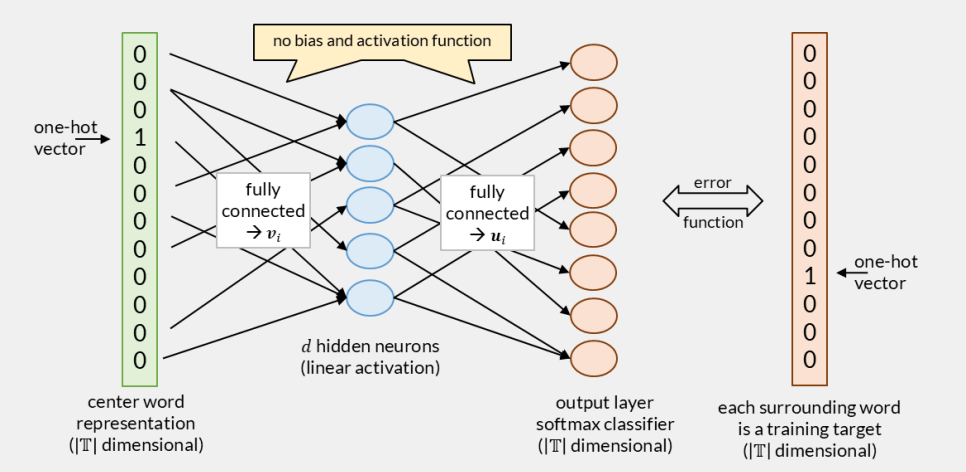


Figure 5.11: Skip-Gram as a single-hidden-layer neural network. The first weight matrix stores center word vectors $\boldsymbol{v}_i$, the second stores context word vectors $\boldsymbol{u}_i$.

CBOW

The Continuous Bag of Words (CBOW) model is a variant of Word2Vec that reverses the prediction direction: given all surrounding words in a context window, it predicts the center word. Where Skip-Gram asks “given the center word, which context words are likely?”, CBOW asks “given these context words, which center word is likely?” (see Figure 5.12).

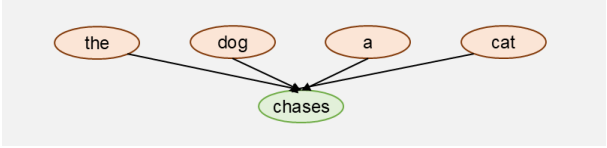


Figure 5.12: CBOW context for "the dog chases a cat": all surrounding words jointly predict the center word "chases".

The key structural difference is how training pairs are formed. Skip-Gram generates $2m$ separate pairs per window, each predicting one context word independently. CBOW generates a single training sample per window: all $2m$ context words are combined into one input to predict the center word. This means CBOW sees each window once, while Skip-Gram processes it $2m$ times.

To combine multiple context words into a single input, CBOW averages their vectors. The probability of the center word w_c given its context uses the same softmax structure as Skip-Gram, but with the averaged context vector replacing the single center word vector:

$$P(w_c \mid w_{c-m}, \ldots, w_{c+m}) = \frac{\exp\left(\boldsymbol{u}_c^\top \cdot \frac{1}{2m}(\boldsymbol{v}_{c-m} + \cdots + \boldsymbol{v}_{c+m})\right)}{\sum_{i \in \mathbb{T}} \exp\left(\boldsymbol{u}_i^\top \cdot \frac{1}{2m}(\boldsymbol{v}_{c-m} + \cdots + \boldsymbol{v}_{c+m})\right)} \tag{5.9}$$

The neural network architecture reflects this difference (Figure 5.13): instead of a single one-hot input, CBOW receives a multi-hot vector (each context word contributes $1/2m$), which the first weight matrix projects into the averaged embedding. From there, the hidden layer and softmax output work identically to Skip-Gram.

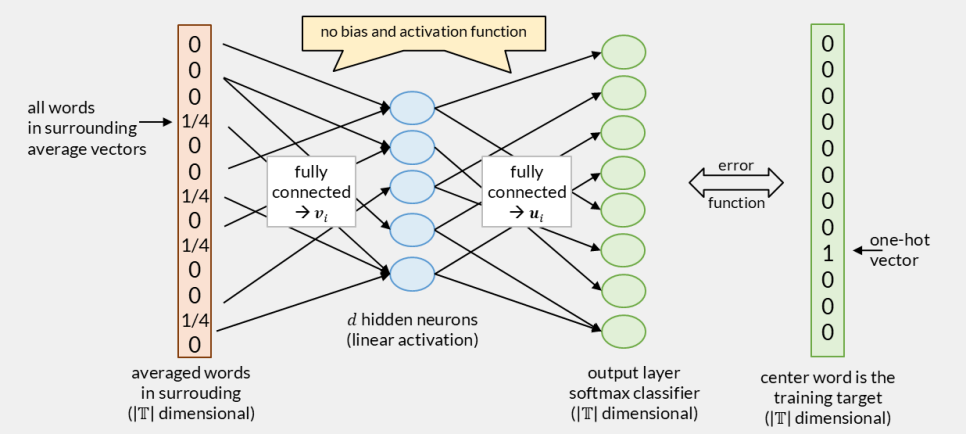


Figure 5.13: CBOW network architecture: context word vectors are averaged into the input, then the same hidden-layer and softmax structure as Skip-Gram predicts the center word.

In practice, the two variants produce embeddings of comparable quality, and neither consistently dominates the other across all benchmarks. The practical differences are:

- **Training speed**: CBOW is faster because it processes one sample per window instead of $2m$.
- **Rare words**: Skip-Gram handles rare words better because each occurrence generates $2m$ gradient updates, giving infrequent words more training signal.
- **Frequent words**: CBOW averages over context, which smooths out noise from very frequent words.

Most practitioners default to Skip-Gram for general-purpose embeddings. The choice rarely matters for downstream retrieval quality; what matters more is corpus size, preprocessing, and hyperparameters (d, m, sub-sampling rate).

Training

A PyTorch implementation of Skip-Gram uses the `Embedding` layer to avoid explicit one-hot vectors. The `Embedding` layer is equivalent to multiplying a one-hot vector by a weight matrix, but works directly with token IDs. The `Linear` layer maps from the hidden dimension back to vocabulary size, producing the logits for the softmax:

```
class SkipGramModel(nn.Module):
    def __init__(self, vocab_size: int, embedd_size: int):
        super().__init__()
        self.embeddings = nn.Embedding(num_embeddings=vocab_size,
embedding_dim=embedd_size)
        self.linear = nn.Linear(in_features=embedd_size, out_features=vocab_size)

    def forward(self, inputs):
        x = self.embeddings(inputs)
        x = self.linear(x)
        return x

# sketch of training process
model = SkipGramModel(vocab_size=len(vocab), embedd_size=100)
optimizer = optim.Adam(model.parameters(), lr=0.001)
loss_fn = nn.CrossEntropyLoss()
for inputs, labels in batch_loader_skipgram(text_corpus):
    optimizer.zero_grad()
    outputs = model(inputs)
    loss = loss_fn(outputs, labels)
    loss.backward()
    optimizer.step()
```

The `batch_loader_skipgram` function generates center-context pairs from overlapping windows across all sentences in the corpus. Each batch contains center word IDs as inputs and context word IDs as labels. `CrossEntropyLoss` combines the softmax and negative log-likelihood into one operation.

Sub-sampling frequent words (reducing pairs containing stop words to $\sim$1% of their original count) improves both quality and speed. The CBOW variant uses the same training loop; the only difference is that the batch loader provides all $2m$ context word IDs as a single input tensor (averaged in the model's forward pass via `x.mean(axis=1)`) and the center word as the label.

GloVe and Subword Models

GloVe

GloVe (Global Vectors for Word Representation, Pennington et al., 2014) also learns word vectors from co-occurrence, but differs from Word2Vec in two ways: it uses a global co-occurrence matrix computed from the entire corpus (rather than streaming through local windows), and its loss function is based on co-occurrence probability ratios rather than prediction accuracy.

Co-occurrence statistics. Let X_{ij} count how often word w_j appears within a context window around word w_i, summed over all occurrences of w_i in the corpus. The total co-occurrence count for w_i is $X_i = \sum_k X_{ik}$, and the probability of word w_j appearing in the context of w_i is $P_{ij} = P(j \mid i) = X_{ij}/X_i$.

The ratio insight. GloVe's key observation is that the *ratio* P_{ik}/P_{jk} reveals semantic relationships more clearly than raw probabilities. To examine whether words w_i and w_j are related, we look at how they each co-occur with a probe word w_k:

- If w_k is related to w_i but not w_j, the ratio P_{ik}/P_{jk} is large.
- If w_k is related to w_j but not w_i, the ratio is small.
- If w_k is related to both or neither, the ratio is close to 1.

The classic example from the original paper uses w_i = "ice" and w_j = "steam":

Probe word w_k	$P(k \mid \text{ice})$	$P(k \mid \text{steam})$	Ratio
solid	1.9×10^{-4}	2.2×10^{-5}	8.9
gas	6.6×10^{-5}	7.8×10^{-4}	0.085
water	3.0×10^{-3}	2.2×10^{-3}	1.36

Probe word w_k	$P(k \mid \text{ice})$	$P(k \mid \text{steam})$	Ratio
fashion	1.7×10^{-5}	1.8×10^{-5}	0.96

"Solid" is strongly associated with "ice" (ratio 8.9), "gas" with "steam" (ratio 0.085), while "water" and "fashion" are equally related or unrelated to both (ratios near 1). Raw probabilities alone do not reveal these distinctions as clearly.

Cost function. GloVe learns word vectors $\boldsymbol{u}_i$ and context vectors $\boldsymbol{v}_j$ (plus biases b_i, c_j) such that the inner product $\boldsymbol{u}_i^\top \boldsymbol{v}_j$ approximates the log co-occurrence count $\log X_{ij}$. This is trained with a weighted least-squares objective:

Key Formula: GloVe Cost Function

$$J = \sum_{i,j=1}^{|\mathbb{T}|} f(X_{ij}) \cdot \left(\boldsymbol{u}_i^\top \boldsymbol{v}_j + b_i + c_j - \log X_{ij}\right)^2 \tag{5.10}$$

The weighting function $f(x) = \min\left((x/x_{\max})^\alpha, 1\right)$ prevents very frequent pairs (like "the"-"of") from dominating the loss. Pairs with $X_{ij} = 0$ are excluded from the sum. Training optimizes $\boldsymbol{u}_i$, $\boldsymbol{v}_j$, b_i, and c_j via gradient descent, similar to Word2Vec.

Note that GloVe reverses the earlier Skip-Gram convention: here $\boldsymbol{u}_i$ is the word vector and $\boldsymbol{v}_j$ the context vector, whereas Skip-Gram and fastText use $\boldsymbol{v}$ for the center word and $\boldsymbol{u}$ for the context; this difference is purely notational, since each model is internally consistent.

The connection to the ratio insight: if the vectors satisfy $\boldsymbol{u}_i^\top \boldsymbol{v}_k \approx \log P_{ik}$, then differences like $\left(\boldsymbol{u}_i - \boldsymbol{u}_j\right)^\top \boldsymbol{v}_k \approx \log(P_{ik}/P_{jk})$ directly encode the co-occurrence ratios that distinguish related from unrelated word pairs.

fastText

fastText (Bojanowski et al., 2017) improves Word2Vec in two ways: sub-word representations for center words, and a more efficient training architecture that replaces the softmax layer.

Sub-word representations. Instead of assigning a single learned vector to each center word, fastText splits the center word into character n-grams and sums their vectors. For the center word "chases" with 3-grams:

$$\boldsymbol{v}_{\text{chases}} = \sum_{g \in \mathcal{Z}} \boldsymbol{z}_g = \boldsymbol{z}_{<\text{ch}} + \boldsymbol{z}_{\text{cha}} + \boldsymbol{z}_{\text{has}} + \boldsymbol{z}_{\text{ase}} + \boldsymbol{z}_{\text{ses}} + \boldsymbol{z}_{\text{es}>} \tag{5.11}$$

The special characters "<" and ">" mark word boundaries, distinguishing prefixes from suffixes from mid-word n-grams. Only the center word is decomposed into sub-words; context words retain their full-word vectors $\boldsymbol{u}_j$. The training still follows the Skip-Gram approach of comparing word pairs (center vs context), but the center word vector is now computed as a sum over its sub-word components rather than looked up directly.

This sub-word approach has two advantages:

1. **Morphological generalization**: "chasing", "chased", and "chases" share sub-words like "cha", "has", so they receive related vectors without explicit stemming.
2. **Handling unknown words**: any new word can be represented by combining its sub-word vectors, even if the full word was never seen during training.

The hands-on notebook makes this concrete: it feeds rare and misspelled terms that are out of vocabulary for GloVe to a fastText model, which still returns sensible neighbors by composing sub-word vectors.

Replacing the softmax. The Skip-Gram softmax over the entire vocabulary $|\mathbb{T}|$ is expensive: each training step requires computing a dot product with every word in the vocabulary for the denominator. fastText replaces this architecture entirely. Instead of a softmax output layer, both the center word and the context word are mapped to their d-dimensional representations, and the model directly compares them

with a dot product. The training objective becomes: make the dot product $\boldsymbol{u}_s^\top \boldsymbol{v}_c$ large for observed pairs and small for non-observed pairs.

To make this work, the model needs negative examples. For each observed center-context pair, fastText samples k negative words (typically $k = 5$ -20) that were not observed in the context of that center word. The loss pushes observed pairs toward high dot products and negative pairs toward low dot products. Choosing good negatives matters: words are sampled proportionally to their frequency raised to the power 3/4, which over-represents rare words relative to uniform sampling. Truly random sampling would often pick words that do co-occur with the center word elsewhere in the corpus; the frequency-weighted approach reduces this risk while keeping computation efficient.

Sub-word granularity. Sub-word lengths can be fixed (3-6 characters) or variable using BPE or WordPiece algorithms, covered under Sub-Word Tokenization later in this chapter. Variable-length sub-words produce more compact vocabularies and handle diverse Unicode scripts better. For instance, English has approximately 3×10^8 possible 6-grams, and storing embeddings for all of them would be impractical. BPE or WordPiece limits the sub-word vocabulary to a manageable size (typically 30k-50k units).

Emergent Properties and Limitations

Word embeddings trained on large corpora exhibit remarkable emergent structure:

- **Semantic clustering**: words with related meanings form clusters in the embedding space.
- **Analogies**: vector arithmetic captures relational patterns. The classic example: $\boldsymbol{v}_{\text{Paris}} - \boldsymbol{v}_{\text{France}} + \boldsymbol{v}_{\text{Germany}} \approx \boldsymbol{v}_{\text{Berlin}}$.
- **Odd-one-out detection**: the word least similar to a group can be identified by distance from the group centroid.

Example

Using pre-trained GloVe vectors (100 dimensions, trained on Wikipedia):

```
import gensim.downloader
wv = gensim.downloader.load('glove-wiki-gigaword-100')

# Most similar to "cat"
wv.most_similar("cat", topn=3)
# [('dog', 0.88), ('rabbit', 0.74), ('cats', 0.73)]

# Analogy: Paris - France + Germany = ?
wv.most_similar(positive=["paris", "germany"], negative=["france"], topn=1)
# [('berlin', 0.88)]

# Odd one out
wv.doesnt_match(["bird", "dog", "cat", "town"])
# 'town'
```

Applying a PCA projection on a few sample words generates the output as shown in Figure 5.14:

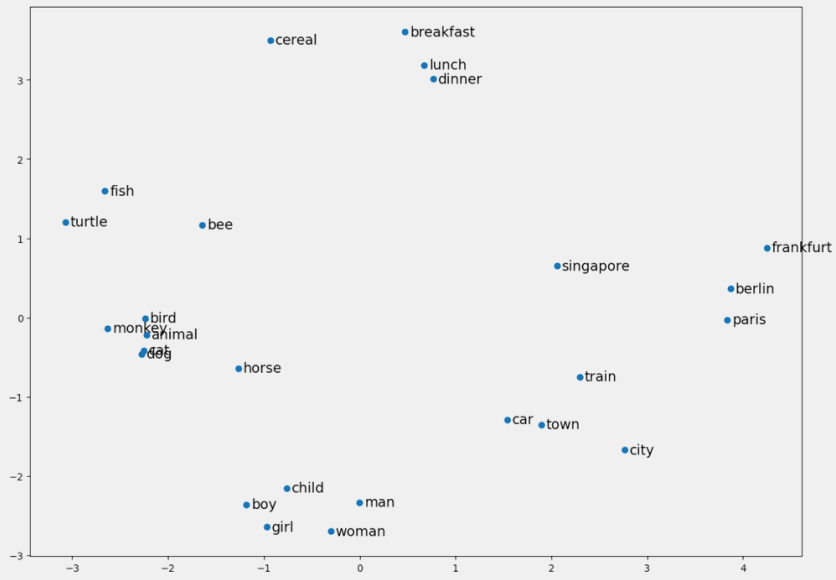


Figure 5.14: PCA projection of GloVe embeddings for 25 words. Semantic clusters emerge: animals (bottom-left), meals (top), cities (right), and people (bottom-center) occupy distinct regions despite no explicit category supervision.

Despite these strengths, static embeddings have a fundamental limitation: **each word receives a single vector regardless of context**. The word "bank" gets the same embedding whether it means a financial institution or a riverbank. In the sentence "I went to the bank on Park Street; I sat and watched the boats go by", humans resolve the ambiguity instantly, but Word2Vec cannot.

Hands-on: Word Embeddings

Explore pre-trained GloVe vectors: inspect nearest neighbors, visualize semantic clusters with PCA, solve analogies through vector arithmetic and see where they fail, compare GloVe with fastText on out-of-vocabulary words, and probe the polysemy limitation of static embeddings. Open notebook ->

Includes pre-run results. You can read through or download and experiment.

From Words to Sentences: Why Pooling Is Not Enough

Word embeddings represent individual words, but semantic search operates on queries and documents that consist of many words. A document cannot be represented as a list of hundreds of separate vectors; we need a single fixed-length vector that captures the meaning of the entire text. The naive approach is to compose word embeddings into a sentence or document embedding through pooling.

Average pooling computes the element-wise mean across all token embeddings. Given n token embeddings $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$, each in $\mathbb{R}^d$:

$$\boldsymbol{e}_{\text{avg}} = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{x}_i \tag{5.12}$$

This produces the geometric centroid of the token vectors. The similarity between a query Q and a document D then reduces to comparing their centroids via cosine similarity. Expanding the dot product reveals what average pooling actually computes: the average pairwise similarity between all query tokens and all document tokens.

Why cosine rather than dot product for static embeddings

For static word embeddings, cosine similarity is the standard choice, not the raw dot product. The reason is that the L2 norm of a learned word vector correlates with the word's corpus frequency, and inversely with its degree of polysemy, so a raw dot product mixes this frequency signal into the comparison

(Schakel and Wilson, 2015). Cosine factors magnitude out and compares only direction, which is the semantic signal we care about. There is a subtlety: Word2Vec optimizes dot products between the center matrix and the context matrix, whereas word-to-word similarity compares two vectors from the same matrix, so neither measure is strictly "the trained one" for this use case, which is part of why the frequency argument decides it. If the vectors are normalized to unit length, cosine and dot product coincide, so the choice only matters for un-normalized vectors; in practice, tools like gensim's `most_similar` normalize first and are therefore effectively using cosine.

Max pooling takes the maximum value in each dimension across all tokens:

$$e_{\max,j} = \max_{1 \le i \le n} x_{i,j} \quad \text{for each dimension } j = 1, ..., d \tag{5.13}$$

This retains the strongest activation in each latent dimension, effectively keeping the most salient features from any token in the sequence.

Why both approaches fail for retrieval. Average pooling dilutes information: in a 500-word document, each token contributes only 0.2% to the final vector. Stop words (which make up 30-50% of natural text) pull the centroid toward generic, uninformative directions in embedding space. The result is that long documents converge toward similar centroids regardless of their actual content, making them hard to distinguish.

Max pooling preserves dominant features but discards relationships between tokens. It cannot distinguish "dog bites man" from "man bites dog" because both contain the same maximum activations. It also ignores which tokens contribute to which dimensions, losing all compositional structure.

Neither method encodes word order, syntactic structure, or long-range dependencies. The sentence "The cat that chased the mouse was black" gets the same pooled representation regardless of whether "black" modifies "cat" or "mouse". For short queries this may be acceptable, but for meaningful retrieval over documents, these limitations are severe.

These shortcomings point to a fundamental architectural need: a model that produces a single vector for an entire sequence while considering how all tokens relate to each other. This is exactly what transformer-based encoders provide, and what the next section introduces.

Transformer Embeddings

Optional Background Reading

This section explains how transformers produce contextual embeddings, and it is here as background. The applications in the next section only need the high-level outcomes, so a reader focused on retrieval can skim or skip it. The next section opens with a short recap of what carries forward.

Word embeddings gave us dense vector representations of individual words, but three fundamental limitations remain before we can build effective neural models for text understanding. This section traces the solutions to each limitation, culminating in the transformer architecture and BERT, the contextual encoder that the next section builds retrieval models on top of.

Three Challenges of Word Embeddings for Neural Models

Word2Vec, GloVe, and fastText produce useful word vectors, but using them as input to deeper neural networks reveals structural problems:

1. **Fixed vocabulary**: Word2Vec and GloVe assign vectors only to words seen during training. A new brand name, a misspelling, or a word from another language has no representation. fastText addresses this with sub-word decomposition, but its character n-grams (3-6 characters) are not linguistically motivated, and the number of possible n-grams is enormous (up to 3×10^8 for English 6-grams), making it impractical as input to a neural network.
2. **Variable vocabulary size**: neural networks require fixed-size input layers. If the vocabulary has 500,000 words, the input layer needs 500,000 dimensions. Changing the vocabulary (adding words, switching language) means rebuilding the model. We need a representation scheme where the vocabulary size is decoupled from the model architecture.
3. **Narrow context window**: Word2Vec operates within a window of 5-10 words, strictly within sentence boundaries. It cannot connect information across sentences. Consider: "The car arrived. It was red and made a lot of noise." The embeddings for the second sentence cannot encode that "red" and "noise" refer to "car", because the context window never reaches back into the first sentence. A retrieval system searching for "noisy red car" would not match the second sentence on its own.

One-Hot Vectors and Why They Fail at Scale

Before solving these problems, we need to understand how discrete tokens are fed into neural networks. The naive approach assigns each token a unique integer ID: "the" arrow.r 1, "cat" arrow.r 2, "and" arrow.r 3, "dog" arrow.r 4. The sentence "the cat and the dog" becomes the sequence [1, 2, 3, 1, 4].

This does not work. Integer IDs impose artificial relationships: if "cat" is 2 and "dog" is 4, the arithmetic $2 \times$ cat = dog is meaningless but implied by the representation. "And" at 3 sits numerically between "cat" and "dog", suggesting a relationship that does not exist.

The standard solution is **one-hot encoding**: represent each token as a vector of length $|\mathbb{T}|$ (the vocabulary size) with all zeros except a single 1 at the position of the token ID. For a vocabulary of 4 words:

$$\text{"the"} = [1,0,0,0], \quad \text{"cat"} = [0,1,0,0], \quad \text{"and"} = [0,0,1,0], \quad \text{"dog"} = [0,0,0,1] \tag{5.14}$$

One-hot vectors are orthogonal: no token is numerically "closer" to another. This removes the false relationships. It is exactly what Skip-Gram uses: the one-hot input selects a row from the weight matrix, which is the embedding vector for that token.

One-hot vectors have two problems. First, they encode term independence: every token is equally different from every other token. But we know from LSI and word embeddings that terms are not independent; semantically related tokens reside in a lower-dimensional subspace. A 500,000-dimensional one-hot space is excessive because the true information content requires far fewer dimensions. LSI solved this with SVD, but we do not want to pay the computational cost of SVD for every new corpus. Instead, we will learn this dimensionality reduction as part of the model training (see learned embeddings below).

Second, the vocabulary size directly determines the input layer size, and vocabularies vary across languages, domains, and corpora. A model trained with 500,000 English words cannot accept input from a 600,000-word corpus without rebuilding the input layer and retraining.

Sub-Word Tokenization

Sub-word tokenization addresses the variable vocabulary problem by fixing the vocabulary size regardless of the corpus. Instead of one token per word, text is split into smaller pieces from a compact vocabulary of a predetermined size (typically 30k-50k tokens). Any word, including one never seen during training, can be decomposed into known sub-word pieces. This is similar to fastText's sub-word idea, but with two differences: the sub-word units are variable in length (not fixed 3-6 character n-grams), and they are determined by a data-driven algorithm that adapts to the frequency distribution of the training corpus.

Byte Pair Encoding (BPE)

BPE builds its vocabulary iteratively:

1. Start with all individual characters as the initial vocabulary.
2. Count the frequency of all adjacent pairs of vocabulary items across the corpus. In the first iteration these are character pairs; in later iterations a "pair" can consist of multi-character tokens from previous merges.
3. Merge the most frequent pair into a new vocabulary entry.
4. Update all word representations by replacing occurrences of that pair with the new token.
5. Repeat steps 2-4 until the desired vocabulary size is reached.

Example

For the sentence "this course is about this topic" (after lowercasing):

Initial vocabulary: `a, b, c, e, h, i, o, p, r, s, t, u`

Word representations with frequencies:

```
this     (×2):   t, h, i, s
course   (×1):   c, o, u, r, s, e
is       (×1):   i, s
about    (×1):   a, b, o, u, t
topic    (×1):   t, o, p, i, c
```

Round 1: adjacent pairs with counts `is` (3), `th` (2), `hi` (2), `ou` (2), `co` (1), … Merge the most frequent pair `i, s` (3 times: twice in "this", once in "is") into token `is`:

```
this     (×2):   t, h, is
is       (×1):   is
```

Round 2: pairs `th` (2), `h·is` (2), `ou` (2), … are now tied at 2; merge `t, h` into `th`:

```
this     (×2):   th, is
```

Round 3: `th·is` (2) and `ou` (2) are tied; merge `th, is` into the whole word `this`:

```
this     (×2):   this
```

Round 4: `ou` (2, in "course" and "about") is the only remaining pair above count 1; merge `o, u` into `ou`:

```
course   (×1):   c, ou, r, s, e
about    (×1):   a, b, ou, t
```

Every remaining adjacent pair now occurs only once, so we stop here (a real run would keep going until the target vocabulary size is reached). Applying the four merges gives the encoding:

```
this | c·ou·r·s·e | is | a·b·ou·t | this | t·o·p·i·c
```

> Only “this” and “is” appear as whole tokens, because the merge sequence actually produced them. “course”, “about”, and “topic” stay split into the sub-word pieces that survive: the only merge reaching them is `ou`, so “course” becomes `c·ou·r·s·e`, “about” becomes `a·b·ou·t`, and “topic” (untouched by any merge) stays `t·o·p·i·c`. Words not in the vocabulary are always decomposed into whatever sub-word pieces exist.

Newer BPE variants operate at the byte level (starting with 256 initial entries), enabling them to handle any Unicode script without reserving the full character set in the vocabulary.

WordPiece

WordPiece follows the same iterative process but differs in two ways:

1. It distinguishes word-initial characters from word-internal ones using a “##” prefix. The starting vocabulary for our example becomes: `a, c, i, t, ##b, ##c, ##e, ##h, ##i, ##o, ##p, ##r, ##s, ##t, ##u`. This preserves prefix information: “un” in “unhappy” becomes a different token from “##un” in “running”, capturing that prefixes often carry shared meaning across words.
2. Instead of merging the most frequent pair, it merges the pair whose components appear together disproportionately often compared to their individual frequencies: $\text{score}(a, b) = \text{tf}(a, b)/(\text{tf}(a) \cdot \text{tf}(b))$. This is the same Pointwise Mutual Information criterion used for n-gram extraction. It prefers pairs that are strongly associated rather than merely frequent. A minimum frequency filter is needed to avoid favoring pairs that appear only once.

SentencePiece

SentencePiece (Kudo and Richardson, 2018) is a language-independent tokenization framework that treats the input as a raw byte stream rather than pre-tokenized words. Unlike BPE and WordPiece, which first split text into words at whitespace and then decompose words into sub-word pieces, SentencePiece operates directly on the character sequence, including spaces (represented as a special “▁” character). This makes it suitable for languages without clear word boundaries (Chinese, Japanese, Thai) and avoids language-specific pre-processing rules.

SentencePiece supports two sub-word algorithms: BPE (same merge-by-frequency strategy as above) and **Unigram**, which takes the opposite approach. Instead of building up from characters, Unigram starts with a large candidate vocabulary and iteratively removes tokens whose removal least reduces the likelihood of the training corpus. The final vocabulary contains tokens that together best explain the corpus under a unigram language model. XLM-RoBERTa and models built on it (such as BGE-M3) use SentencePiece with Unigram and a 250k vocabulary, enabling them to handle over 100 languages.

In practice, all modern language models use some variant of BPE or SentencePiece. WordPiece remains in use only through BERT-based models. Vocabulary sizes range from 30k to 250k tokens. The exact size is a design choice: larger vocabularies represent more words as single tokens (faster encoding, more parameters in the embedding matrix), while smaller vocabularies decompose more aggressively (fewer parameters, but longer token sequences).

Algorithm	Used by	Vocab size	Merge criterion
WordPiece	BERT, MiniLM, Nomic Embed	30k	PMI (association strength)
BPE	GPT, Mistral, Qwen, Kimi	32k-160k	Frequency (most common pair)
SentencePiece (Unigram)	XLM-RoBERTa, BGE-M3	250k	Likelihood (corpus probability)

> **Two meanings of “embedding”**
>
> The term “embedding” is overloaded. (1) The **input embedding layer** maps tokens to vectors as the first step of a neural network. (2) A **semantic embedding** is a vector representation of an entire text passage produced by the full model for retrieval. In this subsection, we discuss the input embedding

(meaning 1). The retrieval embeddings (meaning 2) are the output of the complete architecture discussed later in this section.

Hands-on: Sub-Word Tokenization

Implement Byte Pair Encoding from scratch and watch it reproduce the merge example above, then load the real tokenizers of BERT (WordPiece), GPT-2 (BPE), and XLM-RoBERTa (SentencePiece) to compare how they split text, mark word boundaries, and break rare words into sub-words. Open notebook ->

Includes pre-run results. You can read through or download and experiment.

Learned Embeddings

With a fixed-size sub-word vocabulary, we can map each token to a one-hot vector of manageable size (30k-50k dimensions). But one-hot vectors are still sparse and high-dimensional. The next step is to learn a dense, lower-dimensional representation, exactly as Word2Vec does.

An **embedding layer** is a weight matrix $\boldsymbol{E} \in \mathbb{R}^{|\mathbb{T}| \times d}$ where row j is the d-dimensional embedding for token j. Multiplying a one-hot vector by $\boldsymbol{E}$ simply selects the corresponding row, which is a fast lookup operation. The embedding dimensionality d (typically 768 for BERT-Base) is independent of the vocabulary size.

The key difference from Word2Vec: these embeddings are not trained separately. They are initialized randomly and refined during the training of the full model. The embedding layer is the first component of the transformer architecture; its weights are updated by the same gradient descent that trains the attention layers, so the embeddings co-adapt with the rest of the model.

This solves the variable-vocabulary problem: the embedding matrix is the only component whose size depends on $|\mathbb{T}|$. Everything downstream operates in d dimensions regardless of vocabulary size.

From Sequential to Parallel Context

The remaining challenge is context. Word embeddings are context-free. To understand "bank" in "The bank approved the loan", the model needs to attend to "approved" and "loan". Earlier neural approaches tried to solve this:

Recurrent Neural Networks (RNNs) process tokens one at a time, carrying an internal hidden state that accumulates context from previous tokens. After reading "The", the state captures some information about the first word. After "The bank", it captures both. By the end of the sentence, the hidden state theoretically encodes the full context. LSTMs and GRUs improved on vanilla RNNs by better preserving long-range information.

RNNs had two critical limitations. First, they are inherently sequential: token i cannot be processed until token $i-1$ is finished. This prevents parallelization and makes training on long sequences very slow. Second, despite LSTM improvements, the hidden state still degrades over long distances. In a 500-word document, information from the first sentence is heavily diluted by the time the model reaches the last sentence.

Self-Attention: The Breakthrough

The 2017 paper "Attention Is All You Need" (Vaswani et al.) introduced the transformer, arguably the most consequential architecture in modern AI. Every large language model (GPT, Claude, Gemini, Llama), every modern embedding model (SBERT, GTE, Nomic), and every state-of-the-art retrieval system is built on transformers. The core innovation is **self-attention**: instead of processing tokens sequentially, every token directly attends to every other token in parallel.

Self-attention solves both RNN limitations at once. It processes all positions in parallel (enabling GPU acceleration over long sequences), and every token can directly access every other token regardless of distance (no information degradation over long sequences). The model itself learns which tokens are

relevant to which other tokens, rather than relying on a fixed context window or a sequential accumulation of state.

Concretely, self-attention transforms each token embedding $\boldsymbol{x}_i \in \mathbb{R}^d$ into a new, context-aware representation $\boldsymbol{z}_i \in \mathbb{R}^d$. The output has the same dimensionality as the input, but now "bank" at position 2 will have a different $\boldsymbol{z}_2$ depending on whether the surrounding words are about finance or rivers.

How self-attention works: Q/K/V mechanism (optional reading)

Self-attention computes context-aware representations through three steps: project, score, and aggregate.

1. Project each token embedding $\boldsymbol{x}_i \in \mathbb{R}^d$ into three vectors using learned linear mappings $\boldsymbol{W}_Q \in \mathbb{R}^{d_k \times d}$, $\boldsymbol{W}_K \in \mathbb{R}^{d_k \times d}$, $\boldsymbol{W}_V \in \mathbb{R}^{d_v \times d}$:

$$\boldsymbol{q}_i = \boldsymbol{W}_Q \boldsymbol{x}_i \in \mathbb{R}^{d_k}, \quad \boldsymbol{k}_i = \boldsymbol{W}_K \boldsymbol{x}_i \in \mathbb{R}^{d_k}, \quad \boldsymbol{v}_i = \boldsymbol{W}_V \boldsymbol{x}_i \in \mathbb{R}^{d_v} \tag{5.15}$$

The query $\boldsymbol{q}_i$ is used to compare (via dot product) against the keys $\boldsymbol{k}_j$ of all other tokens: a high dot product $\boldsymbol{q}_i^\top \boldsymbol{k}_j$ means token j is relevant to token i. The value $\boldsymbol{v}_j$ is the contribution that token j makes to the new representation of token i when the attention weight is high. Queries and keys must share the same dimensionality d_k (so the dot product is defined). In practice, $d_k = d_v = d/h$ where h is the number of attention heads. For BERT-Base: $d = 768$, $h = 12$, so each head operates in $d_k = 64$ dimensions.

2. Compute attention weights for token i over all positions j. The raw compatibility between query i and key j is their scaled dot product $\boldsymbol{q}_i^\top \boldsymbol{k}_j / \sqrt{d_k}$. To convert these into a probability distribution, we apply the softmax function, which exponentiates each score and normalizes by the sum:

$$\alpha_{ij} = \text{softmax}_j\left(\frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j}{\sqrt{d_k}}\right) = \frac{\exp\left(\boldsymbol{q}_i^\top \boldsymbol{k}_j / \sqrt{d_k}\right)}{\sum_{l=1}^{n} \exp\left(\boldsymbol{q}_i^\top \boldsymbol{k}_l / \sqrt{d_k}\right)} \tag{5.16}$$

The exponential ensures all weights are positive, and dividing by the sum ensures $\sum_j \alpha_{ij} = 1$. A high α_{ij} means token i attends strongly to token j.

3. Aggregate by attention pooling: each token j contributes its value $\boldsymbol{v}_j$ proportionally to its attention weight, producing the context-aware representation:

$$\boldsymbol{z}_i = \sum_{j=1}^{n} \alpha_{ij} \boldsymbol{v}_j \tag{5.17}$$

Matrix form. All three steps can be computed in one expression using matrices $\boldsymbol{Q}$, $\boldsymbol{K}$, $\boldsymbol{V}$ (whose rows are the individual query, key, and value vectors):

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}\right)\boldsymbol{V} \tag{5.18}$$

Here $\boldsymbol{Q}\boldsymbol{K}^\top$ is an $n \times n$ matrix of all pairwise dot products. The softmax is applied to each row independently (normalizing across columns). Multiplying by $\boldsymbol{V}$ computes all weighted sums in parallel. The division by $\sqrt{d_k}$ prevents dot products from growing too large with increasing dimension, which would push the softmax into near-zero gradients and slow training.

Figure 5.15 illustrates this process for computing $\boldsymbol{z}_2$ ("bank") in "The bank approved the loan". The query for "bank" is compared against all keys; "approved" and "loan" receive the highest attention weights because their keys are most compatible with "bank" in a financial context.

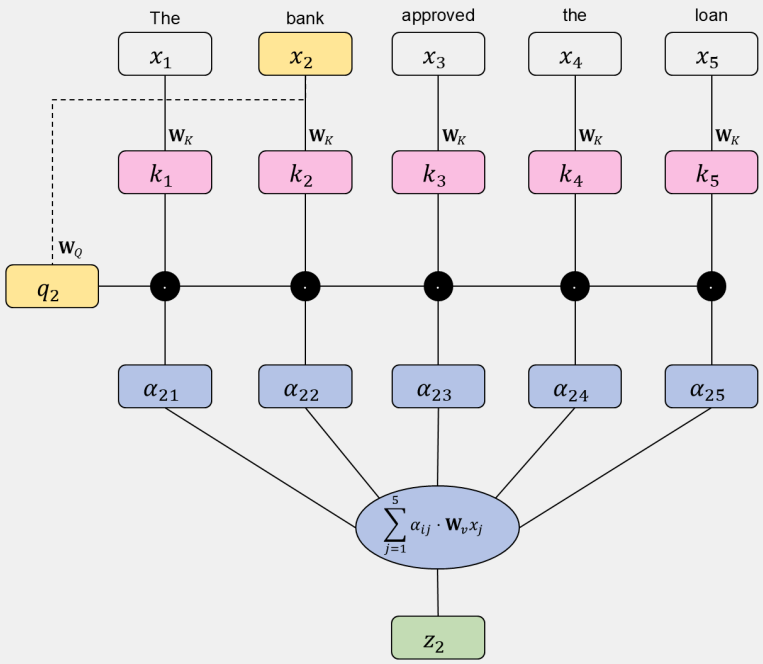


Figure 5.15: Self-attention computing z_2 for "bank". The query q_2 is compared against all keys to produce attention weights. "Approved" (0.52) and "loan" (0.36) dominate, so z_2 is primarily a blend of their values, making "bank" context-aware as a financial institution.

Multi-head attention runs h parallel attention operations with different projection matrices, allowing the model to attend to different types of relationships simultaneously. The outputs are concatenated and projected back to d dimensions:

$$\text{MultiHead}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{Concat}(\text{head}_1, ..., \text{head}_h)\boldsymbol{W}_O \tag{5.19}$$

Each head operates in d/h dimensions; concatenating h heads recovers the full d-dimensional output. Figure 5.16 shows how multi-head attention works within one layer: h parallel heads each run self-attention in d/h dimensions with independent projections, then their outputs are concatenated and projected back to d dimensions.

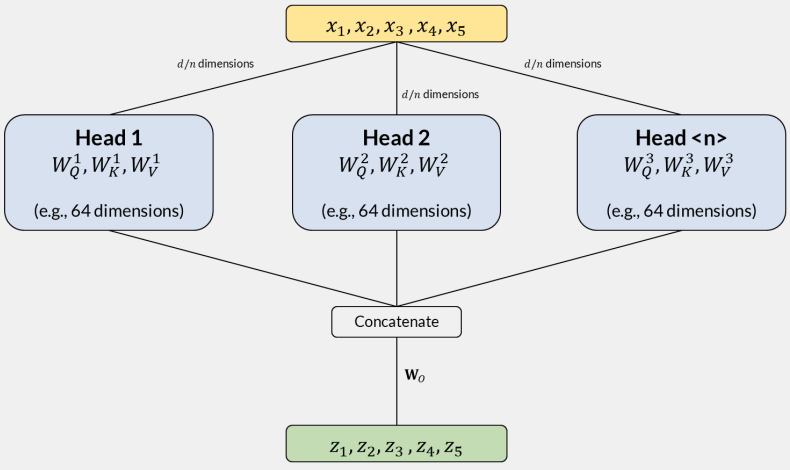


Figure 5.16: Multi-head attention: h parallel heads each run self-attention in d/h dimensions with independent projections. Concatenating all heads recovers the full d-dimensional output.

Positional Encoding

Self-attention is permutation-invariant: the formula $\boldsymbol{z}_i = \sum_j \alpha_{ij} \boldsymbol{v}_j$ depends on token content but not on position. "Dog bites man" and "man bites dog" would produce identical representations without additional

information. To preserve sequence order, a positional encoding $\boldsymbol{p}_i \in \mathbb{R}^d$ is added to each token embedding before the first attention layer:

$$\boldsymbol{x}_i = \boldsymbol{E}[\text{token}_i] + \boldsymbol{p}_i \tag{5.20}$$

The original transformer uses fixed sinusoidal functions at different frequencies for each dimension. BERT uses learned position vectors (one per position up to 512). Both approaches add position information without increasing the embedding dimensionality.

The Transformer Architecture

A single self-attention layer produces one round of contextualization, but it has a fundamental limitation: the aggregation $\boldsymbol{z}_i = \sum_j \alpha_{ij} \boldsymbol{v}_j$ is a weighted sum, linear in the value vectors. While the attention weights themselves are non-linear (due to softmax), stacking multiple attention layers alone would still produce representations that are largely linear combinations of the inputs. To model complex, non-linear relationships between tokens, two additional components are needed.

The **feed-forward network (FFN)** applies a non-linear transformation to each token independently: $\text{FFN}(\boldsymbol{z}) = \text{ReLU}(\boldsymbol{z}\boldsymbol{W}_1 + \boldsymbol{b}_1)\boldsymbol{W}_2 + \boldsymbol{b}_2$. The ReLU (or GELU in newer models) is where the non-linearity enters the transformer. Without it, the model could not learn complex token interactions regardless of depth.

Residual connections add the input of each sub-layer directly to its output ($\boldsymbol{x} + \text{SubLayer}(\boldsymbol{x})$), creating shortcut paths through the network. During backpropagation, gradients flow through these shortcuts without passing through the attention or FFN computations, which prevents vanishing gradients and allows transformers to be stacked 12 to 96 layers deep (this idea originates from ResNets in computer vision). **Layer normalization** normalizes the activations at each layer, stabilizing training and reducing sensitivity to weight initialization.

Together, one transformer layer consists of: multi-head self-attention arrow.r Add & Norm arrow.r feed-forward network arrow.r Add & Norm. Figure 5.17 shows the full architecture with these components stacked into an encoder and a decoder.

The left side is the **encoder**: it processes the full input sequence bidirectionally (each token attends to all others) and produces contextualized representations. The right side is the **decoder**: it generates output tokens one at a time, using masked self-attention (each token can only attend to previous positions) plus cross-attention to the encoder output. For embedding and retrieval, we use only the encoder side.

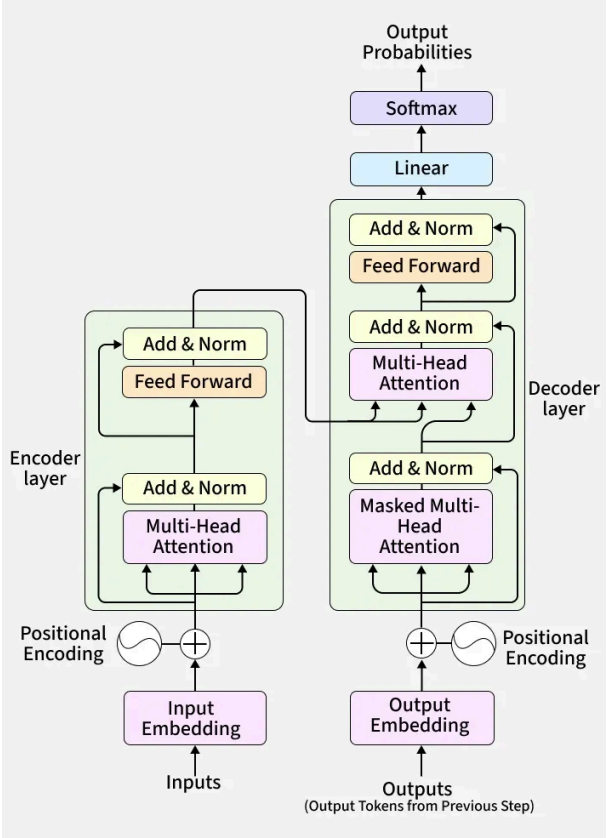


Figure 5.17: The transformer architecture (Vaswani et al., 2017). Left: the encoder processes input bidirectionally through stacked layers of multi-head self-attention and feed-forward networks. Right: the decoder generates output auto-regressively with masked attention. For text embeddings, only the encoder is used.

BERT: Bidirectional Context

BERT (Devlin et al., 2019) applies the transformer encoder to produce contextualized token representations. Unlike GPT, which processes text left-to-right (suitable for generation), BERT attends to context from both directions simultaneously, making it better suited for understanding and representation tasks.

BERT uses WordPiece tokenization, learned positional encodings, and adds one BERT-specific element: **segment encodings** that mark whether a token belongs to sentence A or sentence B when processing sentence pairs. Two special tokens frame the input: `[CLS]` at the start (whose final hidden state serves as a sequence-level representation) and `[SEP]` separating segments. The maximum sequence length is 512 tokens; shorter sequences are padded, longer ones truncated.

Figure 5.18 shows the complete input construction for an example sentence. The three embeddings (token, position, segment) are summed element-wise to form the input to the first encoder layer. BERT-Base stacks 12 encoder layers with 12 attention heads per layer; BERT-Large uses 24 layers with 16 heads.

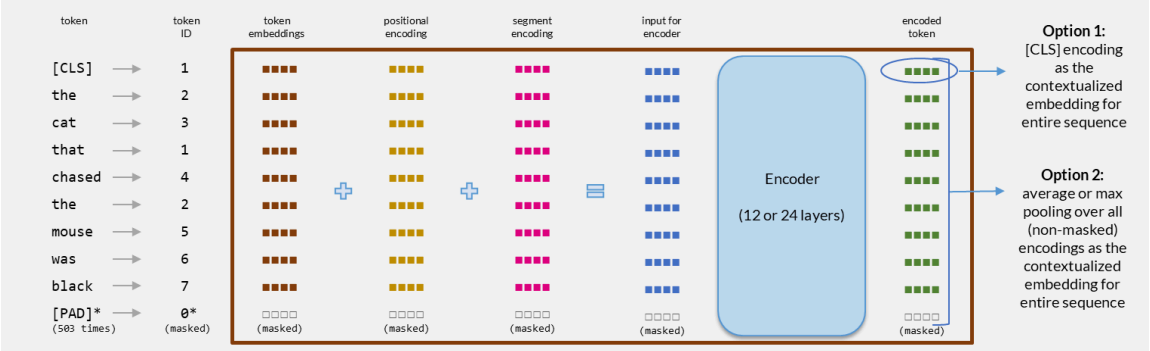


Figure 5.18: BERT input construction for "the cat that chased the mouse was black". Token embeddings, positional encodings, and segment encodings are summed to produce the encoder input. After 12/24 layers, each token has a contextualized output vector. Option 1: use the `[CLS]` encoding as the sequence embedding. Option 2: pool over all non-masked token encodings.

Pre-training

BERT is pre-trained on two self-supervised tasks:

- **Masked Language Modeling (MLM)**: randomly mask 15% of input tokens and predict them from context.
- **Next Sentence Prediction (NSP)**: given two sentences, predict whether the second follows the first in the original text.

After pre-training on large corpora (Wikipedia + BookCorpus), BERT's encoder produces 768-dimensional (Base) or 1024-dimensional (Large) contextualized vectors for each token. The same word receives different representations depending on its surrounding context.

Semantic Similarity with Transformers

What carries over from the previous section

- **Contextual embeddings**: transformers such as BERT produce context-dependent token vectors, so the same word gets a different vector depending on its surrounding words. This is why static word embeddings are not enough for semantic search.
- **One vector per passage**: a transformer yields a sequence-level embedding via the `[CLS]` token or by pooling over the token outputs. That single vector is the input to the retrieval architectures in this section.
- **Sub-word tokenization**: text is split into sub-word units from a fixed-size vocabulary (BPE, WordPiece, or SentencePiece), which handles out-of-vocabulary words and many languages.

For the full background, see Transformer Embeddings.

Those contextual token representations are the starting point here. Turning them into a single comparable vector for retrieval, and comparing two passages accurately, is a separate problem. This section covers the three architectures that solve it: bi-encoders, cross-encoders, and late interaction models.

BERT produces rich contextualized token representations, but these are internal states optimized for the pre-training objectives (predicting masked tokens and sentence adjacency), not for representing the overall meaning of a passage. The `[CLS]` token's hidden state encodes whatever information the Next Sentence Prediction task required, which is not the same as a general semantic summary. Using it directly as a sentence embedding for retrieval produces results worse than simple Word2Vec averaging. Mean pooling over all token outputs is marginally better but still underperforms static embeddings on semantic similarity benchmarks. Extracting useful sentence-level representations from BERT required a different training approach: Sentence-BERT.

Bi-Encoders: Sentence-BERT

Sentence-BERT (Reimers and Gurevych, 2019) restructures BERT into a **bi-encoder** (also called dual-encoder or siamese network): two sentences are encoded independently by the same transformer, then compared via dot product (equivalent to cosine similarity because the embeddings are normalized to unit length). Figure 5.19 contrasts this with the simpler approaches discussed earlier in this chapter.

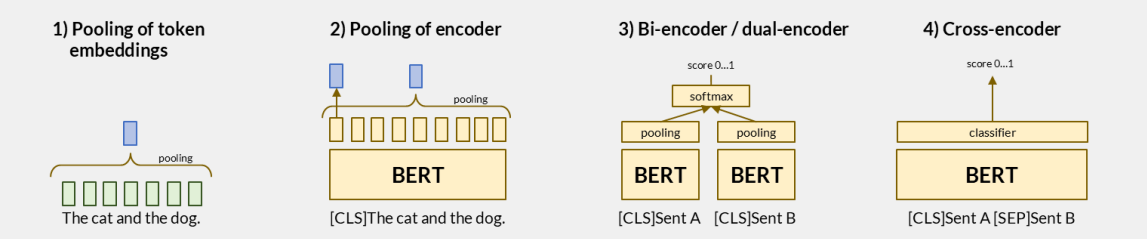


Figure 5.19: Four architectures for sentence similarity, progressing from simple to powerful: (1) pooling static word embeddings, (2) pooling BERT token outputs, (3) bi-encoder with independently encoded sentences compared via learned similarity, (4) cross-encoder that jointly processes both sentences for maximum accuracy.

Which similarity measure for learned sentence embeddings

For static word embeddings the frequency argument settles the question in favor of cosine. For learned sentence embeddings the rule is different: match the measure the model was trained with. Some models are trained with a normalized cosine objective, so their output is unit length and cosine equals dot product. Others are trained for dot-product retrieval, where the vector norm carries meaning and normalizing it away would discard signal. The model card states which objective the model was trained with, and this is the case where "use the measure the model was trained with" is the correct rule.

Architecture and training

The bi-encoder produces fixed-size sentence embeddings:

1. Pass a sentence through BERT (or any transformer encoder).
2. Apply mean pooling over the non-masked token outputs to produce a single d-dimensional vector.
3. Normalize the vector to unit length.

Training uses similarity-based losses on labeled sentence pairs:

- **Contrastive loss**: pulls similar pairs close, pushes dissimilar pairs apart with a margin.
- **Triplet loss**: given an anchor, a positive, and a negative, ensures $\text{sim}(a, p) > \text{sim}(a, n) + \varepsilon$.
- **Multiple negatives ranking loss (MNRL)**: treats all other examples in the batch as negatives, maximizing efficiency of each training step.

Hard negative mining (selecting negatives that are challenging but incorrect) is critical for training high-quality bi-encoders.

Inference efficiency

The bi-encoder's key advantage for retrieval: documents are encoded once during indexing. At query time, only the query needs encoding. Comparison is a dot product (equivalent to cosine similarity for normalized vectors), enabling sub-millisecond ranking over millions of pre-computed embeddings.

Modern bi-encoder variants

Since SBERT, the field has advanced significantly:

- **Instruction-tuned embeddings**: models like E5-Mistral, GTE-Qwen2, and Nomic Embed accept a task instruction prefix (e.g., "Represent this document for retrieval:") that adapts the embedding to the downstream task without retraining.
- **Decoder-based encoders**: Qwen3-Embedding and similar models build on decoder transformers (using the final `[EOS]` token representation instead of `[CLS]`), achieving state-of-the-art quality at the cost of larger models (0.6B to 8B parameters).
- **Extended context**: modern embedding models support 8k-32k token inputs, enabling direct encoding of longer passages without chunking. However, longer context does not come for free: compressing 30k tokens into a single d-dimensional vector loses more information than compressing 512 tokens into the same space. Research on long-context LLMs shows that attention disproportionately favors tokens near the beginning and end of the sequence, losing information from the middle. In practice, chunking a long document into shorter passages and retrieving at chunk level often outperforms encoding the full document as one vector. We return to this tradeoff in the next section.

Cross-Encoders and Reranking

A cross-encoder processes the query and document together as a single input sequence, separated by `[SEP]`:

```
[CLS] query [SEP] document [SEP]
```

The `[CLS]` token's final hidden state passes through a classification layer to produce a relevance score between 0 and 1. Because all tokens from both query and document attend to each other through every transformer layer, the cross-encoder captures fine-grained token-level interactions that bi-encoders miss.

Quality vs efficiency tradeoff

Property	Bi-Encoder	Cross-Encoder
Encoding	Query and document separately	Query and document jointly
Pre-computation	Documents encoded at index time	Every pair must be processed at query time
Complexity per query	$O(1)$ per document (dot product)	$O(n \cdot L)$ per document (L = sequence length)
Semantic depth	Good (independent representations)	Excellent (full token-level interaction)
Typical use	First-stage retrieval	Reranking top-k candidates

Two-stage pipeline

Cross-encoders are too expensive for exhaustive search over large collections. The standard deployment pattern is:

1. **Retrieve**: a bi-encoder (or BM25) selects the top-k candidates (typically $k = 100$ -1000).
2. **Rerank**: a cross-encoder scores each candidate with full token interaction and reorders the list.

This hybrid approach achieves near cross-encoder quality at near bi-encoder speed. The hands-on notebook for this section measures this tradeoff directly: it scores the same challenging sentence pairs with both a bi-encoder and a cross-encoder and compares the wall-clock cost of each.

Late Interaction: ColBERT

ColBERT (Khattab and Zaharia, 2020) occupies the middle ground between bi-encoders and cross-encoders by computing interaction at the token level while still allowing document pre-computation.

Architecture

Unlike bi-encoders (which compress each text into a single vector) or cross-encoders (which process both texts jointly), ColBERT (Figure 5.20):

1. **Encodes** query and document independently through a transformer, retaining all token-level embeddings (not pooling to a single vector).
2. **Interacts** via late interaction: each query token computes maximum similarity against all document tokens.
3. **Scores** by summing these maximum similarities across query tokens:

Key Formula: ColBERT MaxSim Scoring

$$\text{score}(q, d) = \sum_{i=1}^{|q|} \max_{j=1}^{|d|} \boldsymbol{q}_i^\top \boldsymbol{d}_j \tag{5.21}$$

Each query token finds its best-matching document token (MaxSim), and the total score sums these per-token matches. This captures fine-grained term-level alignment without requiring joint encoding.

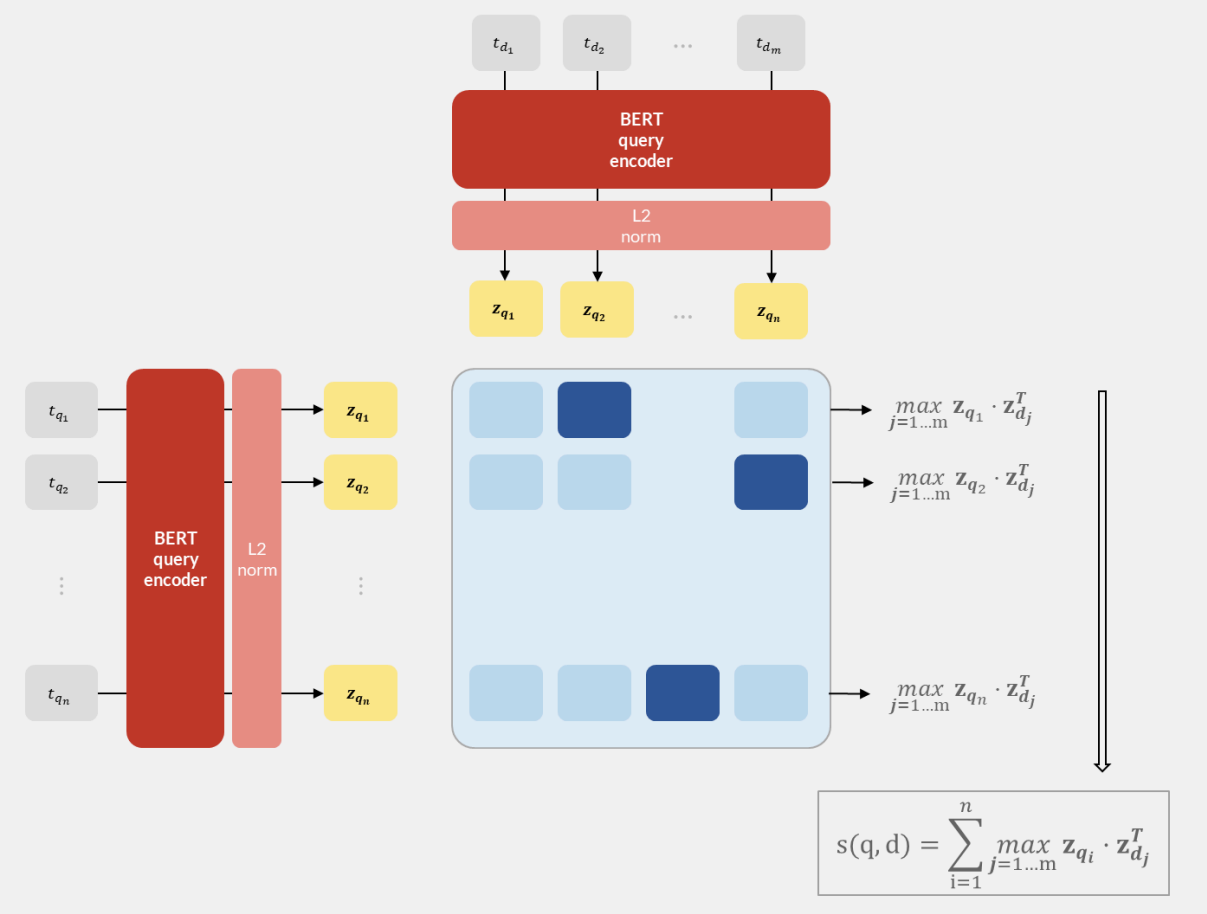


Figure 5.20: ColBERT architecture: query and document are encoded independently (allowing document pre-computation), but scoring uses token-level MaxSim interaction rather than a single-vector dot product.

Training

As with SBERT, raw BERT token outputs are not directly useful for MaxSim comparison. ColBERT uses a shared BERT encoder for both queries and documents, followed by a small linear projection that reduces each token embedding from 768 to 128 dimensions (reducing storage and comparison cost). The full system is trained end-to-end on triples of (query, relevant document, irrelevant document) with a ranking loss that maximizes the MaxSim score for the relevant document and minimizes it for the irrelevant one. Because the MaxSim scoring function is differentiable, gradients flow through it back into the encoder, teaching the model to produce token embeddings where semantically matching query-document token pairs have high dot products.

Why late interaction works

At first glance, ColBERT's per-token matching looks similar to comparing word embeddings. The critical difference is that each token vector is a BERT output, not a static word embedding. Each $\boldsymbol{q}_i$ and $\boldsymbol{d}_j$ has been contextualized by self-attention over the entire query or document. The token "bank" in the document already encodes whether it means a financial institution or a riverbank, based on its surrounding context. When a query token matches a document token via dot product, it is comparing two context-aware representations, not two static word vectors.

This means the MaxSim operation captures phenomena that neither static word embeddings nor single-vector bi-encoders can:

- **Context-sensitive matching**: a query token "treatment" will match strongly with "treatment" in a medical document but weakly with "treatment" in a document about surface treatment of metals, because the BERT outputs differ despite the word being the same.

- **Semantic matching**: a query token "automobile" will have high similarity with the document token "car" because the ranking loss during training explicitly pushed embeddings of semantically related tokens closer together.
- **Partial matching**: not every query token needs to match strongly; the sum allows documents to score well by matching the most important query aspects.

Unlike bi-encoders, ColBERT does not suffer from the compression problem discussed earlier. Each token retains its own vector, so no information is lost by squeezing an entire passage into a single representation. This is why ColBERT can use much lower-dimensional embeddings (128 dimensions per token, projected down from BERT's 768) than bi-encoders (which typically need 768-4096 dimensions per passage): representing one contextualized token requires far fewer dimensions than representing an entire document.

The tradeoff is storage and search cost. A 500-token document produces 500 vectors instead of one. Searching requires computing $|q| \times 500$ dot products per document instead of a single one. Chunking remains useful not for avoiding information compression, but for limiting the number of vectors per unit and keeping semantically related tokens together for more focused matching. ColBERT v2 (Santhanam et al., 2022) addresses the storage cost of retaining per-token embeddings for every document through **residual compression**: token embeddings are quantized by storing only the residual from the nearest centroid, reducing storage by 6-10x while maintaining quality.

Architecture comparison

Architecture	Document representation	Query-time cost	Quality
Bi-encoder	1 vector per document	1 dot product per doc	Good
ColBERT	n vectors per document	$\|q\| \times n$ dot products	Very good
Cross-encoder	None (must re-encode)	Full transformer pass per doc	Excellent

ColBERT achieves 95-98% of cross-encoder quality with 100-1000x faster query processing, making it practical for direct retrieval (not just reranking) over moderately large collections.

Hands-on: Sentence Embeddings and Cross-Encoders

See why averaging word vectors loses sentence meaning, then encode sentences with a bi-encoder (SBERT) and read off a pairwise similarity matrix, watch vocabulary mismatch get resolved, compare bi-encoder against cross-encoder scores and speed on challenging pairs, and step through ColBERT MaxSim scoring on a small example. Open notebook ->

Includes pre-run results. You can read through or download and experiment.

Building Semantic Search

The previous sections traced a progression from BM25 through LSI and word embeddings to transformer-based models. At each step, we reduced the independence assumption between tokens, gaining recall: documents that share no surface terms with the query can now be retrieved through semantic similarity. But this gain comes at three costs that classical retrieval did not face:

1. **Search is expensive.** Dense vectors cannot use inverted indices. Comparing a query against one million document embeddings requires one million dot products. We examine vector search acceleration in the next chapter, but no method matches the pruning efficiency of inverted files for sparse vectors.
2. **Precision drops.** Casting a wider semantic net retrieves more relevant documents, but also more noise. A query for "training large language models" will match papers about "optimizing neural networks" (relevant) and "training dogs with reinforcement signals" (not relevant). Cross-encoders (see Semantic Similarity with Transformers) address this through reranking, but at the cost of a full transformer forward pass per candidate.
3. **Compression loses information.** Encoding a multi-page document into a single 768-dimensional vector discards detail. Chunking the document into shorter passages mitigates this, but multiplies the number of vectors to store and search.

Every design decision in a semantic search pipeline is a tradeoff among three dimensions: **compute cost** (encoding, storage, infrastructure), **latency** (how fast results arrive), and **retrieval quality** (precision and recall for the use case). There is no single optimal pipeline. A factual search engine serving millions of short queries per second operates in a different region of this space than a patent lawyer who needs exhaustive recall and is willing to wait for a thorough assessment.

This section shows how to navigate these tradeoffs through pipeline design, model selection, and efficiency techniques.

Embedding Model Selection

Before designing a pipeline, we need to choose the embedding model that will produce the dense vectors. The Massive Text Embedding Benchmark (MTEB) provides a standardized evaluation framework comparing embedding models across retrieval, semantic textual similarity, classification, and clustering tasks. Retrieval-specific metrics (nDCG@10) are most relevant for search.

Key selection criteria

Criterion	Range	Tradeoff
Embedding dimension	256-4096	Higher dimensions capture more nuance but increase storage and search cost
Model size	33M-8B parameters	Larger models produce better embeddings but require more compute for encoding
Context window	512-32k tokens	Longer windows avoid chunking but compress more content into the same dimensions, degrading quality for long inputs
Multilingual support	1-100+ languages	Multilingual models sacrifice some single-language quality for breadth
Instruction support	Yes/No	Instruction-tuned models adapt to specific tasks via prefixes

Model landscape (2025)

Representative models spanning the quality-efficiency spectrum:

Model	Parameters	Dimensions	Context	MTEB Retrieval (nDCG@10)
all-MiniLM-L6-v2	33M	384	512	~49

Model	Parameters	Dimensions	Context	MTEB Retrieval (nDCG@10)
nomic-embed-text-v1.5	137M	768	8192	~55
jina-embeddings-v3	570M	1024	8192	~58
GTE-Qwen2-7B	7B	3584	32k	~61
Qwen3-Embedding-8B	8B	4096	32k	~63

Note

Benchmark scores evolve rapidly. The specific numbers above reflect mid-2025 results. Always consult the current MTEB leaderboard when making production decisions.

Cost model

To make the cost-latency-quality tradeoff concrete, consider encoding and searching a collection of 100,000 research papers (averaging 6,000 tokens each, chunked into passages of ~500 tokens, yielding roughly 1.2 million chunks):

Operation	Small model (33M)	Large model (8B)
Encode 1.2M chunks	~15 min (1 GPU)	~8 hours (1 GPU)
Storage (float32)	~1.7 GB (384d)	~18 GB (4096d)
Storage (int8 quantized)	~0.4 GB	~4.5 GB
1 query: brute-force dot product	~2 ms	~15 ms
1 query: cross-encoder rerank top-100	~0.5 s (33M)	~10 s (8B)
1 query: BM25 over inverted index	~1 ms	~1 ms

These numbers are approximate and hardware-dependent, but the ratios are instructive: BM25 is orders of magnitude faster than dense search, and cross-encoder reranking is orders of magnitude slower than retrieval. This cost structure drives the pipeline architectures below. The hands-on notebook for this section benchmarks these pipelines on a 500-movie collection and compares a small and a larger embedding model on ranking quality and cost (latency and storage), making these tradeoffs concrete.

Pipeline Architectures

The Figure 5.2 presented the retriever-ranker blueprint and five pipeline variants (A through E). Here we examine why each exists and when it fails, following the natural progression as requirements grow.

Pipeline A (BM25 only) is the baseline from classical retrieval: an inverted index with BM25 scoring, no semantic component. It is fast, well-understood, and sufficient when queries and documents share vocabulary. The subsequent pipelines add semantic components to address its limitations.

Starting simple: BM25 + reranker (Pipeline B)

The simplest semantic search system adds a cross-encoder reranker on top of existing BM25 infrastructure (Figure 5.21). BM25 retrieves the top-k candidates (typically $k = 100$ -1000), and the cross-encoder reorders them by semantic relevance.

This works well when BM25 recall is sufficient: if the relevant documents contain at least some of the query terms, BM25 will find them, and the cross-encoder improves the ordering. It fails when the vocabulary mismatch is severe. A search for "efficient training of large language models" will miss a paper titled "Scaling Laws for Neural Network Optimization" because BM25 cannot connect these terms.

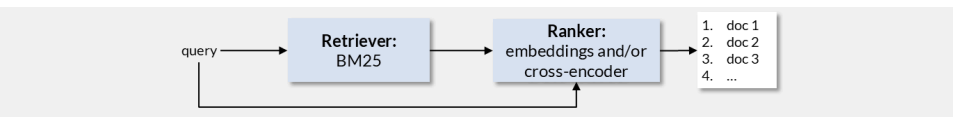


Figure 5.21: Pipeline B: BM25 retrieves candidates from the inverted index, then a semantic ranker (embeddings and/or cross-encoder) reorders them.

Adding dense retrieval: semantic search (Pipeline C)

To overcome the recall ceiling of BM25, we replace it with a bi-encoder that retrieves candidates from a vector index (Figure 5.22). The "Scaling Laws" paper is now retrieved because its embedding is semantically close to the query.

Dense retrieval finds documents that BM25 misses, but it has the opposite weakness: exact-match queries fail. A search for error code "NullReferenceException" or product ID "A1B2C3" depends on exact string matching; the bi-encoder may return semantically similar but wrong results.

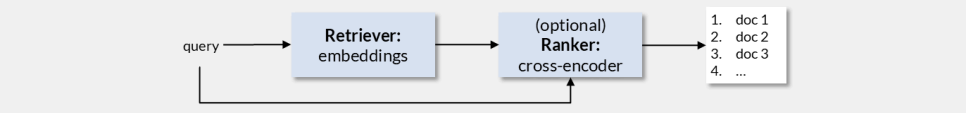


Figure 5.22: Pipeline C: a bi-encoder retrieves candidates from a vector index by semantic similarity, with an optional cross-encoder for reranking.

Going hybrid: sparse + dense (Pipeline D)

The most robust approach runs both retrievers in parallel and fuses the results (Figure 5.23):

1. BM25 retrieves top-k_1 candidates (handles exact matches, rare terms, proper nouns).
2. Dense retrieval retrieves top-k_2 candidates (handles semantic matches, paraphrases).
3. **Reciprocal Rank Fusion (RRF)** merges both result lists.
4. A cross-encoder reranks the merged top-k for final ordering.

RRF assigns each document a fused score based on its rank in each result list:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \tag{5.22}$$

where R is the set of result lists and k is a constant (typically 60) that dampens the influence of high ranks. Documents appearing in both lists receive contributions from both, naturally boosting results that both methods agree on.

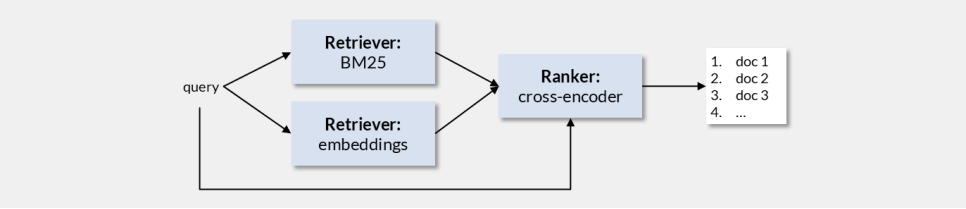


Figure 5.23: Pipeline D: BM25 and dense retrieval run in parallel on the same query. Their candidate lists are fused, then a cross-encoder reranks the merged results.

Chunking for different retrievers

All three pipelines require splitting long documents into shorter passages. The optimal chunk size differs by component:

- **BM25** tolerates large chunks (full sections, even full documents). It scores individual terms within the chunk; surrounding irrelevant text does not affect the score of matching terms. Traditional retrieval systems typically chunk at the page or section level.
- **Bi-encoders** need shorter chunks (256-1024 tokens). The entire chunk is compressed into a single vector, so irrelevant content dilutes the embedding. Paragraph-level or sliding-window chunking with overlap produces the best results.
- **Cross-encoders** can handle longer inputs (up to their context window) because they read query and document jointly with full token interaction. They are typically applied to the chunks retrieved by the earlier stages.

In hybrid systems, a common approach is to chunk at a granularity that works for the bi-encoder (the most sensitive component) and use the same chunks for BM25. The result metadata links chunks back to their source documents so that the final output can present either chunk-level or document-level results. We discuss chunking strategies (fixed-size, recursive, semantic, hierarchical) in detail in the RAG chapter, where the interaction between chunk design and generation quality adds further constraints.

Efficiency at Scale: Multi-Stage Retrieval (Pipeline E)

As collections grow from thousands to millions of documents, the cost of encoding, storing, and searching becomes the dominant concern. Pipeline E addresses this with **multi-stage retrieval**: a cheap first stage scans the full collection for fast filtering, then a precise full-precision second stage reranks the surviving candidates. The two subsections below are variants of this pipeline. They differ only in how the cheap first stage is built: Matryoshka embeddings truncate the vector to a low dimension, while binary or scalar quantization compress each dimension into fewer bits.

Matryoshka Representation Learning (MRL)

Traditional embeddings use a fixed number of dimensions. Matryoshka embeddings (Kusupati et al., 2022) train the model so that truncated prefixes of the embedding remain useful: the first 64 dimensions capture coarse semantics, the first 128 add detail, and the full vector provides maximum precision. The training objective adds loss terms at multiple truncation levels:

$$\mathcal{L}_{\text{MRL}} = \sum_{d \in \mathcal{D}} \mathcal{L}_d\left(\boldsymbol{x}_{[:d]}, \boldsymbol{y}_{[:d]}\right) \tag{5.23}$$

where $\mathcal{D}$ is a set of target dimensions (e.g., {64, 128, 256, 512, 1024}) and $\boldsymbol{x}_{[:d]}$ denotes the first d components of the embedding.

This gives the first variant of Pipeline E, built entirely **within a single model** (Figure 5.24). The truncated prefix serves as the cheap first stage:

1. Use truncated embeddings (e.g., 128 dimensions) for fast coarse retrieval over the full collection.
2. Use full embeddings (e.g., 1024 dimensions) to rerank the top candidates with higher precision.

A cross-encoder can still be appended as a final stage, but the two embedding stages already share one model, so no additional index or encoder is needed.

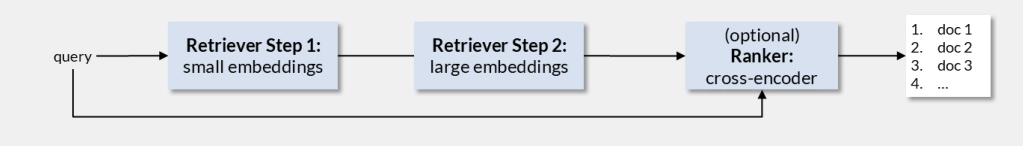


Figure 5.24: Multi-stage retrieval with Matryoshka embeddings: fast initial retrieval using small (truncated) embeddings narrows candidates, then full-dimension embeddings rerank with higher precision. A single model serves both stages.

Binary and scalar quantization

The second variant of Pipeline E builds the cheap first stage through quantization rather than truncation. A compressed (quantized) index handles the fast filtering over the full collection, then the surviving candidates are rescored with the full-precision embeddings. Storage and comparison costs can be further reduced through quantization:

- **Binary quantization**: replace each float dimension with a single bit (positive to 1, negative to 0). Reduces storage by 32x and enables fast Hamming distance computation. Quality loss is typically 5-10% on retrieval benchmarks.
- **Scalar quantization** (int8): map each float to an 8-bit integer. Reduces storage by 4x with minimal quality loss (typically less than 2%).

Quantization is most effective as a first-stage filter: use quantized embeddings for coarse candidate retrieval, then rescore with full-precision embeddings.

Practical Recommendations

There is no single best pipeline. The right design depends on where your application sits in the cost-latency-quality space. The following recommendations serve as starting points, not prescriptions.

By collection size

Collection	Recommended pipeline	Rationale
< 10k documents	Pipeline C (bi-encoder + cross-encoder)	Brute-force search is fast enough; no need for BM25 infrastructure or ANN indices
10k - 1M documents	Pipeline D (hybrid + cross-encoder)	BM25 handles exact match; dense retrieval handles semantic match; cross-encoder refines
> 1M documents	Pipeline E (multi-stage: Matryoshka or quantized first stage)	Multi-stage retrieval keeps latency manageable; ANN index required (see next chapter)

By use case

Use case	Priority	Design emphasis
Factual search (Q&A, support)	Low latency, moderate recall	Small bi-encoder (33M-137M), short chunks, fast reranker
Patent search, legal discovery	High recall, thoroughness	Large bi-encoder (7B+), hybrid retrieval, aggressive cross-encoder reranking, accept higher latency
E-commerce product search	Exact match + semantic, low latency	Hybrid mandatory (product IDs need BM25); lightweight reranker or no reranker
Academic literature search	Broad semantic recall, multilingual	Multilingual model (BGE-M3, Qwen3), hybrid retrieval, moderate reranking

Connection to later chapters

Dense embeddings create the vectors; Chapter 6 (Vector Search) addresses how to efficiently index and search them at scale using approximate nearest neighbor algorithms (HNSW, IVF, product quantization). Chapter 7 (RAG) builds on the retrieval pipeline by feeding retrieved chunks to a language model for answer generation, adding further constraints on chunk design and result quality.

Hands-on: Benchmarking Semantic Search Pipelines

Build the five pipelines (BM25, BM25 with a cross-encoder reranker, dense retrieval, hybrid fusion with Reciprocal Rank Fusion, and multi-stage retrieval) on a 500-movie collection, compare a small and a larger embedding model, and evaluate them against LLM-judged relevance using MRR together with latency and storage. The notebook also shows the Matryoshka recall-versus-storage curve, and why a small model plus BM25 can nearly match a far more expensive large model. Open notebook ->

Includes pre-run results. You can read through or download and experiment.

Summary

Model Comparison

This chapter presented a progression of methods for representing text semantically. Each generation addressed specific limitations of its predecessors while introducing new tradeoffs:

Method	Representation	Context	Transfer	Key Limitation
LSI (1988)	Dense document vectors via SVD	Global (entire corpus)	Corpus-specific	No inverted index; expensive SVD
Word2Vec (2013)	Dense word vectors	Local window (5-10 words)	Language-level	One vector per word (no polysemy)
GloVe (2014)	Dense word vectors	Global co-occurrence ratios	Language-level	One vector per word (no polysemy)
fastText (2017)	Sub-word vectors	Local window + sub-words	Handles OOV words	Still no contextual meaning
BERT (2019)	Contextualized token vectors	Full sequence (512 tokens)	Task-specific fine-tuning	Internal states, not sentence embeddings
Bi-encoder / SBERT (2019)	Single sentence/ document vector	Full sequence	Pre-trained, adaptable	Compresses entire passage into one vector
Cross-encoder (2019)	Relevance score (no vector)	Joint query+document	Task-specific	Too expensive for first-stage retrieval
ColBERT (2020)	Per-token vectors (128d)	Full sequence	Pre-trained	Higher storage than bi-encoders
Matryoshka (2022)	Multi-resolution vectors	Full sequence	Pre-trained	Training complexity

The field has converged on hybrid pipelines: BM25 for exact match and fast candidate retrieval, bi-encoders for semantic recall, and cross-encoders for precision-critical reranking. ColBERT offers a middle path when single-vector bi-encoders lack sufficient quality but cross-encoder latency is unacceptable. Building Semantic Search organizes these components into five reference pipelines (A through E), from a pure BM25 baseline to multi-stage retrieval that pairs a cheap first stage with a precise rerank.

Key Takeaways

1. **The vocabulary mismatch problem** is the fundamental motivation for semantic search: related terms that share no surface forms cannot be matched by keyword methods alone.
2. **Dimensionality reduction discovers latent structure**: whether via SVD (LSI) or neural training (embeddings), mapping text to a compact space forces the model to group co-occurring patterns into latent topics or semantic directions.
3. **Word embeddings learn meaning from context**: the distributional hypothesis drives Word2Vec (Skip-Gram, CBOW) and GloVe, while fastText adds sub-word composition for morphology and out-of-vocabulary words. Transformer-era sub-word tokenizers (BPE, WordPiece, SentencePiece) extend the same idea with a fixed-size vocabulary; their internals are optional background.
4. **Static embeddings cannot handle polysemy, and pooling is not enough**: one vector per word gives "bank" a single representation regardless of context, and averaging or max-pooling word vectors into a passage vector dilutes long documents and discards word order. This is the bridge problem that motivates contextual encoders.

5. **Context transforms meaning**: transformer encoders produce a different vector for the same word depending on its neighbours, solving the multiple-meanings problem that static embeddings cannot. The internal mechanism (self-attention) is optional background; what carries forward to retrieval is the contextual token vectors and the single passage vector obtained by pooling them or reading the `[CLS]` token.
6. **Bi-encoders enable scalable retrieval**: encoding documents once and comparing via dot product achieves sub-millisecond search over millions of documents. Choose the similarity measure deliberately: cosine for static word embeddings (the vector norm tracks frequency and polysemy), and the measure the model was trained with for learned sentence embeddings (cosine when the output is normalized, dot product when the norm carries signal).
7. **Cross-encoders provide precision**: joint encoding of query and document captures fine-grained token interactions, producing the highest quality relevance scores at the cost of a full transformer pass per candidate.
8. **Late interaction (ColBERT)** preserves token-level detail while keeping documents pre-computable. Each token retains its own 128-dimensional vector, avoiding the compression problem of single-vector bi-encoders.
9. **Hybrid retrieval is robust**: combining BM25 (exact match, rare terms) with dense retrieval (semantic matching) via Reciprocal Rank Fusion covers failure modes of either method alone.
10. **Every pipeline is a tradeoff** among compute cost, latency, and retrieval quality. The five reference pipelines (A through E) span this space, from a pure BM25 baseline (A) to multi-stage retrieval (E) that pairs a cheap first stage (Matryoshka truncation or quantization) with a precise full-precision rerank. There is no single best pipeline; collection size and use case decide.

Key Formulas

Key Formula: LSI Projection

$$\bar{\boldsymbol{d}}^\top = \boldsymbol{d}^\top \boldsymbol{U}_k \boldsymbol{S}_k^{-1} \tag{5.24}$$

Maps a term-frequency vector into the k-dimensional latent topic space.

Key Formula: Skip-Gram Probability

$$P(w_s \mid w_c) = \frac{\exp(\boldsymbol{u}_s^\top \boldsymbol{v}_c)}{\sum_{i \in \mathbb{T}} \exp(\boldsymbol{u}_i^\top \boldsymbol{v}_c)} \tag{5.25}$$

Probability of context word w_s given center word w_c, parameterized by embedding vectors.

Key Formula: ColBERT MaxSim Scoring

$$\text{score}(q, d) = \sum_{i=1}^{|\,q\,|} \max_{j=1}^{|\,d\,|} \boldsymbol{q}_i^\top \boldsymbol{d}_j \tag{5.26}$$

Each query token's best document-token match is summed for the final relevance score.

Key Formula: Reciprocal Rank Fusion

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \tag{5.27}$$

Fuses ranked lists from multiple retrievers; documents ranked highly by both methods receive the highest fused scores.

Exam focus

- Explain why LSI can retrieve documents that share no terms with the query (latent topic structure via SVD)
- Contrast Skip-Gram and CBOW: which predicts context from center, which predicts center from context, and why CBOW is faster
- Why averaging or max-pooling static word vectors fails as a passage representation, and what contextual encoders provide instead
- Choosing the similarity measure: cosine for static word embeddings (the norm tracks frequency and polysemy), and matching the training objective (cosine when normalized, dot product otherwise) for learned sentence embeddings
- Why BERT's `[CLS]` state and mean-pooled BERT tokens make poor sentence embeddings out of the box (optimized for the pre-training objectives, not a semantic summary), and how SBERT fixes this
- Bi-encoder vs cross-encoder vs late interaction (ColBERT): the cost-latency-quality tradeoff each represents, and what MaxSim captures that a single-vector dot product cannot
- Why hybrid retrieval (BM25 + dense, fused with RRF) is more robust than either method alone
- The five reference pipelines (A through E): how collection size and use case select among them within the cost-latency-quality space

Self-Check Questions

1. **(Understand)** Why does truncating the SVD to k dimensions bring semantically related terms closer together, even if they never co-occur in the same document?
2. **(Understand)** Explain why Word2Vec assigns the same embedding to "bank" regardless of whether it appears in a financial or geographic context. What property of transformer-based embeddings solves this?
3. **(Understand)** Averaging word embeddings to represent a 500-word document produces weak retrieval results. Give two distinct reasons why, and state what a contextual encoder does differently.
4. **(Analyze)** A bi-encoder encodes a 500-word document into a single 768-dimensional vector. What information is necessarily lost compared to the per-token representations? Give a concrete example of a query that might fail because of this compression.
5. **(Analyze)** Compare the computational cost of ranking 10,000 documents using: (a) a bi-encoder, (b) a cross-encoder, (c) ColBERT with MaxSim. Express each in terms of the number of transformer forward passes required.
6. **(Evaluate)** Your system uses BM25 retrieval followed by cross-encoder reranking with $k = 100$. A user reports that relevant documents for the query "ML model deployment best practices" are not appearing in results, despite existing in the collection. Diagnose the most likely failure point and propose a fix.
7. **(Evaluate)** You are designing a semantic search system for a multilingual legal corpus (5 million documents, 12 languages). Justify your choice of embedding model, chunking strategy, and retrieval architecture using the cost-latency-quality framework.
8. **(Apply)** Given Matryoshka embeddings with dimensions in {64, 128, 256, 512, 1024}, design a two-stage retrieval pipeline. Specify which dimension you use at each stage and why.

Test Your Knowledge

Take the Chapter 5 Quiz ->

Further Reading

- Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). **Indexing by Latent Semantic Analysis**. *Journal of the American Society for Information Science*, 41(6), 391-407. Semantic Scholar The foundational paper introducing LSI and demonstrating that SVD on term-document matrices captures latent semantic structure.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). **Distributed Representations of Words and Phrases and their Compositionality**. *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:1310.4546 Introduces negative sampling and phrase-level Skip-Gram, making Word2Vec practical at scale.

- Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). **Attention Is All You Need**. *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:1706.03762 Introduces the transformer architecture with self-attention, replacing recurrent models and enabling the entire subsequent generation of language models and embedding models.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). **BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding**. *Proceedings of NAACL-HLT 2019*. arXiv:1810.04805 Introduces bidirectional pre-training with masked language modeling, establishing the encoder architecture used by most embedding models.
- Reimers, N. & Gurevych, I. (2019). **Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks**. *Proceedings of EMNLP-IJCNLP 2019*. arXiv:1908.10084 The paper that made transformer-based sentence embeddings practical for retrieval by introducing the bi-encoder training framework with contrastive loss.
- Khattab, O. & Zaharia, M. (2020). **ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT**. *Proceedings of SIGIR 2020*. arXiv:2004.12832 Introduces the late interaction paradigm (MaxSim), achieving near cross-encoder quality with much lower latency by retaining per-token embeddings.
- Kusupati, A., Bhatt, G., Rege, A., et al. (2022). **Matryoshka Representation Learning**. *Advances in Neural Information Processing Systems (NeurIPS 2022)*. arXiv:2205.13147 Trains embedding models to produce useful representations at any truncation level, enabling multi-resolution retrieval from a single model.
- Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). **MTEB: Massive Text Embedding Benchmark**. *Proceedings of EACL 2023*. arXiv:2210.07316 Establishes the standard benchmark for evaluating embedding models across retrieval, similarity, classification, and clustering tasks.